
Formalización de Lógica de Incorrectitud de Separación en F^*



TESINA DE GRADO

LICENCIATURA EN CIENCIAS DE LA COMPUTACIÓN

AUTORA: MARÍA AGUSTINA TORRANO

LEGAJO: T-2989/1

DIRECTOR: GUIDO MARTÍNEZ

5 DE AGOSTO DE 2026

Índice general

Resumen	4
Agradecimientos	5
Preliminares	6
1. Introducción	8
1.1. Motivación	8
1.2. Objetivos del trabajo	9
1.3. Estructura de la tesina	9
2. Conceptos previos	10
2.1. Lógica de Incorrectitud (IL)	10
2.2. Lógica de Separación (SL)	17
2.3. Lógica de Incorrectitud de Separación (ISL)	21
2.4. El lenguaje F^*	26
3. Formalización de IL	30
3.1. Lenguaje y semántica operacional	30
3.2. La IL como tipo inductivo	34
3.3. La Regla de Consecuencia	36
3.4. El Teorema de Soundness	37
4. Formalización de ISL	40
4.1. Extensión del lenguaje y del modelo de estado	40
4.2. Álgebra de <i>heaps</i> y aserciones espaciales	41
4.3. Semántica operacional y metateoría	43
4.4. La ISL como tipo inductivo	45
4.5. El Teorema de Soundness	46
4.6. El Teorema de la Huella	48
5. Evaluación	56
5.1. Detección de errores espaciales básicos (<i>Use After Free</i>)	56
5.2. Reasignación de memoria y punteros colgantes	58
5.3. No determinismo y razonamiento paramétrico sobre bucles	59
5.4. Recorrido de una lista enlazada	61
5.5. Resumen de los resultados	63

6. Conclusiones	65
6.1. Resumen de aportes	65
6.2. Trabajo relacionado	66
6.3. Trabajo futuro	69

Resumen

Desde los inicios de la verificación formal de software, la atención se ha centrado en asegurar la correctitud de los programas, es decir, la ausencia total de errores. Con este fin, sistemas como la Lógica de Hoare y la Lógica de Separación proveen las bases matemáticas para demostrar formalmente que un programa se comporta según su especificación. Sin embargo, la aplicación de estos sistemas a programas de gran tamaño requiere mucho esfuerzo de especificación y prueba.

Frente a estas limitaciones, la industria ha usualmente preferido usar herramientas como analizadores estáticos, herramientas que hacen un mejor esfuerzo para encontrar errores de forma automática. A diferencia de los demostradores de correctitud, estos analizadores suelen emplear abstracciones y aproximaciones de los comportamientos posibles del programa perdiendo precisión en pos de escalabilidad. Aunque estas técnicas pueden estar respaldadas por fundamentos formales, suelen reportar muchos falsos positivos, es decir, advertencias sobre errores que en realidad no son alcanzables en ninguna ejecución real del programa. Esto reduce la confianza y productividad de los desarrolladores, quienes deben invertir tiempo en descartar manualmente errores mal advertidos.

La Lógica de Incorrectitud surge como propuesta para invertir el paradigma tradicional al centrarse en la demostración de errores alcanzables, garantizando la ausencia de falsos positivos en los errores reportados. Recientemente, esta teoría se combinó con el razonamiento local sobre la memoria de la Lógica de Separación, introduciendo la Lógica de Incorrectitud de Separación. Sin embargo, actualmente no existe una formalización mecanizada de esta lógica.

Presentamos la primera formalización de la Lógica de Incorrectitud de Separación utilizando F^* , un lenguaje de programación de tipos dependientes y asistente de pruebas. Por el lado teórico, demostramos las metapropiedades fundamentales de la lógica dentro del asistente de pruebas, haciendo especial énfasis en probar su solidez (*soundness*) matemática para asegurar que cada error detectado sea verdaderamente alcanzable. Por el lado concreto, utilizamos la lógica formalizada para demostrar la existencia de errores en programas no triviales que hacen uso de estado mutable.

Agradecimientos

Agradezco a ...

Preliminares

A lo largo de este trabajo se asume que el lector posee familiaridad con la lógica de primer orden, teoría elemental de conjuntos, relaciones binarias, funciones y fundamentos de la verificación formal de programas, en particular de la Lógica de Hoare (HL). A continuación repasamos brevemente las nociones que se utilizan en los capítulos siguientes, con el fin de fijar la notación y facilitar la comparación entre la Lógica de Hoare y la Lógica de Incorrectitud (IL).

Sea Σ el conjunto de estados posibles de un programa. Una aserción sobre estados es un predicado $p : \Sigma \rightarrow \text{Prop}$. Semánticamente, una aserción puede identificarse con el conjunto de estados que la satisfacen:

$$\llbracket p \rrbracket = \{\sigma \in \Sigma \mid p(\sigma)\}$$

En lo que sigue se utilizará esta identificación de manera implícita. Por lo tanto, las expresiones $\sigma \in p$ y $p(\sigma)$ se considerarán equivalentes.

El comportamiento de un programa se representa mediante una relación binaria sobre estados $R \subseteq \Sigma \times \Sigma$. La pertenencia $(\sigma, \sigma') \in R$ indica que el estado σ' puede obtenerse a partir del estado σ mediante la transición representada por R .

A partir de esta relación y una aserción, podemos definir la imagen posterior de p mediante R como:

$$\text{post}(R)(p) = \{\sigma' \in \Sigma \mid \exists \sigma \in p. (\sigma, \sigma') \in R\}$$

La expresión $\text{post}(R)(p)$ contiene exactamente los estados que pueden alcanzarse mediante una transición de R desde algún estado que satisface p .

Para un comando C , denotaremos su relación semántica mediante $\llbracket C \rrbracket \subseteq \Sigma \times \Sigma$. Por abreviación escribiremos:

$$\text{post}(C)(p) = \text{post}(\llbracket C \rrbracket)(p)$$

A lo largo del trabajo se distinguirán dos niveles de razonamiento. La noción $\vdash_{\mathcal{L}} S$ indica que la especificación S puede derivarse mediante las reglas de inferencia del sistema deductivo $\mathcal{L}$. En cambio, $\models_{\mathcal{L}} S$ indica que S es válida respecto de la semántica del lenguaje.

La HL, introducida por Hoare [10], es un sistema formal para mostrar la correctitud de programas respecto de una especificación. Su unidad básica es la tripleta de Hoare:

$$\{p\} \ C \ \{q\}$$

donde p y q son aserciones (predicados sobre el estado del programa) y C es un comando.

Esta notación establece que si el programa C se ejecuta a partir de un estado que satisface la precondición p , y dicha ejecución termina, entonces el estado resultante satisface la postcondición q . Esta es una correctitud parcial, ya que la tripleta no garantiza que la ejecución vaya a terminar, sino que, en caso de terminar, el resultado será el esperado. Semánticamente:

$$\models_{\text{HL}} \{p\} \ C \ \{q\} \iff \text{post}(C)(p) \subseteq q$$

Esta es la noción de sobre-aproximación, q puede describir más estados de los que el programa realmente alcanza, pero nunca menos. Cuando este principio se utiliza en analizadores automáticos orientados a buscar errores, esos estados pueden dar lugar a falsos positivos.

También se asume familiaridad con la Regla de Consecuencia de la HL, que permite debilitar la postcondición y fortalecer la precondition de una tripleta ya demostrada:

$$\frac{p' \implies p \quad \{p\} \ C \ \{q\} \quad q \implies q'}{\{p'\} \ C \ \{q'\}}$$

Finalmente, se asume familiaridad con los conceptos básicos de programación funcional y teoría de tipos, como tipos inductivos, recursión estructural, cuantificación existencial y universal. Estos son los pilares sobre los que se apoya F^* , el lenguaje utilizado en este trabajo.

Capítulo 1

Introducción

“The price of reliability is the pursuit of the utmost simplicity.”

— C. A. R. Hoare

En este primer capítulo se presenta el propósito general del trabajo. En primer lugar, se expone la motivación que dio origen a la tesina y el problema que se busca abordar. Luego, se formulan sus objetivos generales. Finalmente, se describe la organización del documento y se resume el contenido de cada uno de los capítulos.

1.1. Motivación

La verificación formal busca establecer propiedades sobre el comportamiento de los programas mediante modelos y demostraciones matemáticas. Con los trabajos fundacionales de Floyd [7] y Hoare [10], una parte importante de esta disciplina se ha concentrado en demostrar que los programas satisfacen sus especificaciones y, como consecuencia, que determinadas clases de errores no pueden producirse. A partir de estas primeras propuestas, se desarrollaron numerosos sistemas lógicos capaces de verificar propiedades cada vez más complejas, incluyendo concurrencia, *liveness* y uso de recursos.

Los métodos deductivos como la Lógica de Hoare (HL) y la Lógica de Separación (SL) [19], permiten demostrar que un programa satisface una especificación, aunque suelen requerir un gran esfuerzo de anotación y prueba. En cambio, los analizadores estáticos priorizan la automatización y la escalabilidad, pero las aproximaciones que utilizan pueden reducir su precisión y dar lugar a falsos positivos.

Para proporcionar garantías rigurosas sobre los errores reportados, estos análisis deben tener base en algún marco teórico apropiado y bien entendido. Con este propósito, O’Hearn [16] introdujo la Lógica de Incorrectitud (IL), una propuesta reciente que invierte el paradigma tradicional al centrarse en la demostración de errores alcanzables, aportando una base teórica sólida para imposibilitar los falsos positivos.

La IL se basa en la sub-aproximación, sus postcondiciones describen únicamente estados que pueden obtenerse mediante ejecuciones reales. En consecuencia, si el sistema permite derivar un error, existe una traza de ejecución que conduce hasta él. La lógica puede no representar todos los comportamientos posibles y, por lo tanto, puede omitir ciertos errores, pero los errores demostrados no constituyen falsos positivos.

La IL es análoga a la HL para incorrectitud. Siendo así, sufre de limitaciones similares a la hora de razonar sobre programas con estado mutable y estructuras de datos complejas. En el ámbito de la correctitud, la SL fue desarrollada como una extensión a la HL que soporta razonamiento composicional sobre programas con estados complejos. Recientemente, Raad

et al. [17] introdujeron la Lógica de Incorrectitud de Separación (ISL), cambiando el foco hacia incorrectitud pero manteniendo la modularidad brindada por la SL.

Las definiciones y metapropiedades de una lógica pueden demostrarse matemáticamente sobre un papel. Sin embargo, estos razonamientos pueden contener errores u omisiones difíciles de identificar. Una forma de incrementar la confianza en estos resultados consiste en mecanizarlos dentro de un asistente de pruebas, de modo que cada definición y cada paso de una demostración sean comprobados por el sistema. En esta tesina se usa F^* , un lenguaje de programación orientado a pruebas que combina tipos dependientes, especificaciones lógicas y automatización mediante el solucionador SMT Z3.

La formalización desarrollada no constituye un analizador automático completo ni implementa un procedimiento general para descubrir errores de manera autónoma. Las derivaciones, las precondiciones y los caminos de ejecución relevantes son construidos explícitamente, mientras que F^* verifica que cada paso sea matemáticamente válido. El trabajo se concentra en establecer un núcleo lógico mecanizado que pueda servir como base para el desarrollo futuro de herramientas automáticas de detección de errores.

1.2. Objetivos del trabajo

El objetivo general de esta tesina es formalizar la ISL en el asistente de pruebas F^* y demostrar su solidez respecto de una semántica operacional explícita. Esta formalización se desarrolla de manera incremental.

En una primera etapa se mecaniza la IL sobre un lenguaje imperativo simplificado, sin operaciones de memoria dinámica. Esta primera formalización permite estudiar de forma aislada las características propias de la sub-aproximación, el no determinismo, la composición secuencial y el razonamiento sobre bucles.

En una segunda etapa se extiende el modelo para incorporar memoria mutable y razonamiento espacial. Para ello se redefine el estado del programa mediante un *store*, un *heap* y un modo de terminación, y se incorporan operaciones de lectura, escritura, liberación y reserva de memoria.

Además, se busca que la formalización mantenga una separación entre el núcleo lógico de la ISL y la sintaxis del lenguaje imperativo utilizado. Esto pretende facilitar futuras extensiones del modelo y su posible adaptación a lenguajes más expresivos. Sin embargo, la reutilización directa del núcleo para distintos lenguajes concretos y la construcción de un analizador automático completo queda fuera del alcance de esta tesina.

1.3. Estructura de la tesina

El resto de la tesina se organiza de la siguiente manera.

El Capítulo 2 presenta los conceptos teóricos necesarios para el desarrollo del trabajo. Se introducen la IL, la SL, la ISL y las características principales de F^* necesarias para entender la mecanización.

El Capítulo 3 desarrolla la formalización de la IL. Se definen el lenguaje imperativo, su semántica operacional y la demostración de su solidez.

El Capítulo 4 extiende esta formalización para incorporar memoria dinámica. Se presenta el modelo de *heaps*, aserciones espaciales, *heaps* negativos, la conjunción separadora, la Regla del Marco y la demostración de la solidez de la ISL.

El Capítulo 5 presenta distintos ejemplos verificados mediante la formalización, incluyendo casos relacionados con errores de memoria, no determinismo, bucles y estructuras enlazadas.

Finalmente, el Capítulo 6 resume los resultados obtenidos, los trabajos relacionados, las limitaciones de la propuesta y posibles líneas de trabajo futuro.

Capítulo 2

Conceptos previos

“Program testing can be quite effective for showing the presence of bugs, but is hopelessly inadequate for showing their absence.”

— Edsger W. Dijkstra

En este capítulo se describen los fundamentos teóricos de las lógicas formalizadas a lo largo de este trabajo, abordando sus conceptos centrales y reglas de inferencia. Asimismo, se presenta una introducción a F^* , el lenguaje y asistente de pruebas utilizado para el desarrollo práctico.

En primer lugar, se expone la Lógica de Incorrectitud, seguida por la Lógica de Separación, detallando las definiciones formales y las particularidades de cada sistema. Luego, se introduce la Lógica de Incorrectitud de Separación.

2.1. Lógica de Incorrectitud (IL)

Desde los inicios de la verificación formal, el enfoque predominante ha sido garantizar la ausencia de errores en el software. Sin embargo, en el desarrollo de software a escala industrial, los programadores pasan gran parte de su tiempo razonando sobre código incorrecto y buscando errores. La Lógica de Incorrectitud (IL), introducida por O’Hearn [16], propone lo inverso a la Lógica de Hoare. En lugar de razonar sobre la ausencia de errores, provee las bases para demostrar matemáticamente la presencia de los mismos mediante la sub-aproximación.

La principal diferencia entre la IL y la HL se encuentra en la dirección de la aproximación utilizada. Mientras que la HL emplea sobre-aproximaciones para abarcar todos los comportamientos posibles de un programa, la IL utiliza sub-aproximaciones que describen únicamente comportamientos respaldados por ejecuciones reales.

Antes de presentar las tripletas de incorrectitud, es necesario observar cómo se interpretan sus aserciones. Sea Σ el conjunto de estados posibles del programa. Una aserción p es un predicado sobre estados, es decir, una función $p : \Sigma \rightarrow \mathbf{Prop}$. Equivalentemente, puede identificarse con el conjunto de estados que la satisfacen:

$$\llbracket p \rrbracket = \{\sigma \in \Sigma \mid p(\sigma)\}$$

De ahora en más, utilizaremos indistintamente una aserción y el conjunto de estados que representa escribiendo $p \subseteq \Sigma$.

La semántica de un comando puede representarse mediante una relación binaria entre estados. Para un comando C y una condición de salida $\epsilon \in \{ok, er\}$, denotamos por

$$\llbracket C \rrbracket_\epsilon \subseteq \Sigma \times \Sigma$$

la relación que contiene exactamente los pares (σ, σ') tales que C puede ejecutarse desde el estado inicial σ y finalizar en σ' mediante el canal de salida ϵ .

Definición 2.1 (Post-imagen). Sea $r \subseteq \Sigma \times \Sigma$ una relación entre estados y sea $p \subseteq \Sigma$ una aserción. La post-imagen $post(r) \in P(\Sigma) \rightarrow P(\Sigma)$ de p mediante r se define como:

$$post(r)p = \{\sigma' \mid \exists \sigma \in p. (\sigma, \sigma') \in r\}$$

La post-imagen contiene exactamente los estados que pueden alcanzarse mediante la relación r partiendo de algún estado que satisface p .

Aplicando esta definición para un comando C y una condición de salida ϵ , definimos:

$$post_\epsilon(C)(p) = post(\llbracket C \rrbracket_\epsilon)(p)$$

Por lo tanto, $post_\epsilon(C)(p) = \{\sigma' \mid \exists \sigma \in p. (\sigma, \sigma') \in \llbracket C \rrbracket_\epsilon\}$.

Lema 2.2. (*Propiedades de la post-imagen*) Sea $r \subseteq \Sigma \times \Sigma$ una relación y sean $p, p_1, p_2 \subseteq \Sigma$. Entonces:

$$p_1 \subseteq p_2 \implies post(r)(p_1) \subseteq post(r)(p_2)$$

y

$$post(r)(p_1 \cup p_2) = post(r)(p_1) \cup post(r)(p_2)$$

Demostración. La primera propiedad se sigue directamente de que todo estado perteneciente a p_1 también pertenece a p_2 . Para la segunda, un estado pertenece a la post-imagen de $p_1 \cup p_2$ si posee un estado inicial en p_1 o en p_2 , lo que equivale a pertenecer a alguna de las dos post-imágenes. ■

La post-imagen representa el conjunto exacto de estados alcanzables. Tanto la HL como la IL comparan sus resultados con este conjunto, pero lo hacen en direcciones opuestas.

Definición 2.3 (Tripletas semánticas). Sean $r \subseteq \Sigma \times \Sigma$ una relación y $p, q \subseteq \Sigma$ dos aserciones. La tripleta sub-aproximada:

$$[p] \ r \ [q] \text{ es válida si y solo si } q \subseteq post(r)(p)$$

La tripleta sobre-aproximada o tripleta de Hoare:

$$\{p\} \ r \ \{q\} \text{ es válida si y solo si } post(r)(p) \subseteq q$$

La diferencia entre ambas tripletas reside en la dirección de la aproximación. La tripleta de Hoare busca describir un conjunto que contenga todos los estados alcanzables, mientras que la tripleta sub-aproximada describe solo estados cuya alcanzabilidad puede justificarse.

La relación entre ambas aproximaciones puede resumirse como:

$$q_{sub} \subseteq post(r)(p) \subseteq q_{sobre}$$

En consecuencia, una sobre-aproximación imprecisa puede incluir comportamientos que no se producen en ninguna ejecución real y dar lugar a falsos positivos. Una sub-aproximación puede omitir comportamientos y, por lo tanto, no encontrar todos los errores posibles, pero no debe introducir estados finales imposibles.

La tripleta de incorrectitud se obtiene instanciando la tripleta sub-aproximada con la semántica de un comando y una condición de salida.

Definición 2.4. (Tripleta de incorrectitud) Sean C un comando, $p, q \subseteq \Sigma$ dos aserciones y $\epsilon \in \{ok, er\}$ una condición de salida. Una tripleta de incorrectitud se escribe:

$$[p] \ C \ [\epsilon : q]$$

La tripleta es semánticamente válida, lo que denotamos mediante:

$$\models_{IL} [p] \ C \ [\epsilon : q]$$

si y solo si $q \subseteq post_\epsilon(C)(p)$, donde p se denomina presunción, mientras que q describe un resultado para el canal de terminación ϵ . El canal ok representa terminación normal y el canal er una provocada por un error.

Lema 2.5. (Caracterización) *Los siguientes son equivalentes:*

1. $\models_{IL} [p] \ C \ [\epsilon : q]$.
2. *Todo estado que satisface q puede alcanzarse desde algún estado que satisface p : $\forall \sigma_q \in q. \exists \sigma_p. (\sigma_p, \sigma_q) \in \llbracket C \rrbracket_\epsilon$.*

Demostración. Por la Definición 2.4,

$$\models_{IL} [p] \ C \ [\epsilon : q] \iff q \subseteq post_\epsilon(C)(p)$$

Luego, con la Definición 2.1, podemos afirmar que esta inclusión equivale a exigir que, para todo $\sigma_q \in q$, exista un estado $\sigma_p \in p$ tal que

$$(\sigma_p, \sigma_q) \in \llbracket C \rrbracket_\epsilon$$

■

La caracterización anterior explica la utilidad de la IL para el análisis automático de programas. Si una herramienta demuestra una postcondición de error satisfacible, cada estado descrito por ella está respaldado por una ejecución real. De esta forma, la lógica evita reportar como errores estados finales que no sean alcanzables.

Esta garantía se vuelve especialmente relevante en analizadores automáticos aplicados a bases de código extensas. En ese contexto, una herramienta puede emitir miles de advertencias, y cada falso positivo requiere que un desarrollador inspeccione manualmente una ejecución que en realidad no puede producirse. Muchas falsas alarmas no solo incrementan el costo de revisión, sino que también reducen la confianza en el analizador y pueden provocar que se ignoren reportes válidos.

La IL no constituye por sí misma un algoritmo para descubrir errores, sino un marco lógico para justificar la solidez de analizadores orientados a encontrarlos. Una herramienta puede utilizar heurísticas para seleccionar caminos, sintetizar presunciones o explorar el código, mientras que la lógica establece qué debe demostrarse para garantizar que cada error reportado corresponde a una ejecución real.

Derivabilidad y reglas de inferencia

Las definiciones anteriores caracterizan la validez de una tripleta respecto de la semántica del programa. Sin embargo, demostrar directamente la inclusión $q \subseteq post_\epsilon(C)(p)$ puede ser difícil. Por esta razón, la IL proporciona un sistema deductivo compuesto por reglas de inferencia que permiten construir tripletas a partir de la estructura de los comandos.

Definición 2.6. (Derivabilidad en la IL) Escribimos

$$\vdash_{\text{IL}} [p] \ C \ [\epsilon : q]$$

cuando la tripleta puede obtenerse mediante una derivación finita construida utilizando las reglas de inferencia de la IL.

La derivabilidad es una noción sintáctica, determinada por las reglas del sistema. En cambio, la validez es una noción semántica, definida mediante la post-imagen. La relación entre ambas nociones será establecida por el Teorema de Soundness. Las reglas de inferencia se presentan en la Figura 2.1.

CONSECUENCIA $\frac{p \implies p' \quad \vdash_{\text{IL}} [p] \ C \ [\epsilon : q] \quad q' \implies q}{\vdash_{\text{IL}} [p'] \ C \ [\epsilon : q']}$		DISYUNCIÓN $\frac{\vdash_{\text{IL}} [p_1] \ C \ [\epsilon : q_1] \quad \vdash_{\text{IL}} [p_2] \ C \ [\epsilon : q_2]}{\vdash_{\text{IL}} [p_1 \vee p_2] \ C \ [\epsilon : q_1 \vee q_2]}$
SKIP $\vdash_{\text{IL}} [p] \ \text{skip} \ [ok : p]$	SECUENCIA (ERROR) $\frac{\vdash_{\text{IL}} [p] \ C_1 \ [er : r]}{\vdash_{\text{IL}} [p] \ C_1; C_2 \ [er : r]}$	SECUENCIA (NORMAL) $\frac{\vdash_{\text{IL}} [p] \ C_1 \ [ok : q] \quad \vdash_{\text{IL}} [q] \ C_2 \ [\epsilon : r]}{\vdash_{\text{IL}} [p] \ C_1; C_2 \ [\epsilon : r]}$
KLEENE (BASE) $\vdash_{\text{IL}} [p] \ C^* \ [ok : p]$	KLEENE (PASO) $\frac{\vdash_{\text{IL}} [p] \ C^*; C \ [\epsilon : q]}{\vdash_{\text{IL}} [p] \ C^* \ [\epsilon : q]}$	KLEENE (VARIANTE) $\frac{\vdash_{\text{IL}} [p(n) \wedge \text{nat}(n)] \ C \ [ok : p(n+1) \wedge \text{nat}(n)]}{\vdash_{\text{IL}} [p(0)] \ C^* \ [ok : \exists n. p(n) \wedge \text{nat}(n)]}$
CHOICE ($i = 1$ o 2) $\frac{\vdash_{\text{IL}} [p] \ C_i \ [\epsilon : q]}{\vdash_{\text{IL}} [p] \ C_1 + C_2 \ [\epsilon : q]}$	ERROR $\vdash_{\text{IL}} [p] \ \text{error} \ [er : p]$	ASSUME $\vdash_{\text{IL}} [p] \ \text{assume}(B) \ [ok : p \wedge B]$
ASIGNACIÓN $\vdash_{\text{IL}} [p] \ x = e \ [ok : \exists x'. p[x'/x] \wedge x = e[x'/x]]$	NONDET $\vdash_{\text{IL}} [p] \ x = \text{nondet} \ [ok : \exists x'. p]$	

Figura 2.1: Reglas de inferencia de la IL. Adaptadas de [16].

Las reglas de CONSECUENCIA y DISYUNCIÓN reflejan propiedades generales de la post-imagen. La regla de CONSECUENCIA permite debilitar la precondition y fortalecer la post-condición para achicar el espacio de estados. La regla de la DISYUNCIÓN permite combinar derivaciones obtenidas independientemente desde diferentes conjuntos de estados iniciales. Esto se corresponde con el hecho de que la post-imagen preserva uniones, establecido en el Lema 2.2.

SKIP proporciona las aserciones esperadas para la terminación normal, es decir, no existe una ejecución de **skip** que finalice en el canal de error. Por el contrario, en la regla ERROR todo el estado va al canal anormal (*er*).

Para el caso de secuenciación hay dos reglas. La primera regla (SECUENCIA (ERROR)) se refiere al cortocircuito, que se da cuando se produce un error dentro del primer comando. En este caso, el segundo comando no se ejecuta. La segunda (SECUENCIA (NORMAL)), se da cuando el primer comando finaliza normalmente y se puede continuar con el segundo.

La regla CHOICE determina que para alcanzar un estado basta con que una de las ramas del operador no determinista $+$ sea capaz de lograrlo. La lógica no exige analizar ambas cuando el objetivo es justificar una ejecución concreta. Por su parte, ASSUME filtra el estado mediante una proposición B en el canal de éxito, sin generar errores.

Las mutaciones de las variables operan hacia adelante. ASIGNACIÓN introduce una variable lógica existencial x' para retener el valor histórico previo de x evitando pérdidas de precisión. De forma análoga, la regla NONDET sobrescribe la variable con un valor indeterminado eliminando sus restricciones previas en la postcondición.

El lenguaje utilizado incluye además el operador de estrella de Kleene C^* . Este representa la repetición no determinista y finita del comando C , el cuerpo puede ejecutarse cero, una o cualquier cantidad finita de veces. No expresa por sí mismo la condición de un bucle **while**; un bucle condicionado puede codificarse mediante:

$$(\text{assume } (B); C)^*; \text{assume } (\neg B).$$

Luego, el análisis de los bucles se descompone en tres reglas. La regla KLEENE (BASE) corresponde a la salida inmediata del bucle. La regla KLEENE (PASO) utiliza estructuralmente la forma $C^*; C$ en lugar de $C; C^*$ para facilitar el razonamiento en casos donde se lanza un error dentro de una iteración. En este escenario, pasarán iteraciones exitosas y el error se disparará en la última de ellas. Finalmente, la regla KLEENE (VARIANTE) introduce el principio de inducción matemática sobre variables naturales para generalizar y propagar hacia adelante estos estados alcanzables que incrementan de forma predecible en cada vuelta.

La Regla de Consecuencia y la sub-aproximación

Una de las diferencias lógicas más contraintuitivas de la IL respecto a la verificación clásica es el comportamiento de su Regla de Consecuencia. En correctitud, la Regla de Consecuencia permite olvidar información debilitando la postcondición para cubrir todos los caminos posibles, por ejemplo, eliminando conjunciones lógicas:

$$\{p\} \ C \ \{q_1 \wedge q_2\} \implies \{p\} \ C \ \{q_1\}$$

Esto es válido porque, si todos los estados alcanzables satisfacen $q_1 \wedge q_2$, también satisfacen q_1 .

En la IL, debilitar la postcondición de esta forma no es válido. El sistema exige retener el contexto exacto del camino evaluado para garantizar que los estados sigan siendo alcanzables. En su lugar, la sub-aproximación permite descartar caminos enteros de ejecución basándose en su propia Regla de Consecuencia. Esta se define de la siguiente manera:

$$\frac{p \implies p' \quad \vdash_{\text{IL}} [p] \ C \ [\epsilon : q] \quad q' \implies q}{\vdash_{\text{IL}} [p'] \ C \ [\epsilon : q']}$$

Interpretando las aserciones como conjuntos, las condiciones laterales equivalen a $p \subseteq p'$ y $q' \subseteq q$. Por lo tanto, la regla permite ampliar el conjunto de estados iniciales y reducir el conjunto de estados finales.

La dirección de estas inclusiones puede justificarse utilizando las definiciones anteriores y el Lema 2.2. Si la tripleta original es semánticamente válida, entonces:

$$q \subseteq \text{post}_\epsilon(C)(p)$$

Como $p \subseteq p'$, por monotonía de la post-imagen:

$$\text{post}_\epsilon(C)(p) \subseteq \text{post}_\epsilon(C)(p')$$

Además, $q' \subseteq q$. Por transitividad:

$$q' \subseteq q \subseteq \text{post}_\epsilon(C)(p) \subseteq \text{post}_\epsilon(C)(p')$$

Por lo tanto:

$$q' \subseteq \text{post}_\epsilon(C)(p')$$

que es la validez semántica de la nueva tripleta.

En particular, si se sabe que la ejecución de un condicional alcanza los estados de múltiples ramas, por ejemplo $q_1 \vee q_2$, sabiendo que $q_1 \implies q_1 \vee q_2$, la Regla de Consecuencia avala descartar el segundo término de la postcondición:

$$\frac{\vdash_{\text{IL}} [p] \ C \ [\epsilon : q_1 \vee q_2] \quad q_1 \implies q_1 \vee q_2}{\vdash_{\text{IL}} [p] \ C \ [\epsilon : q_1]}$$

Esta capacidad que tiene la IL es fundamental. Permite que las herramientas de análisis automático enfoquen su exploración en trazas específicas y omitan el resto.

Ejemplo 2.1. *Considere el siguiente fragmento de código y la presunción inicial $p = \{\sigma \in \Sigma \mid \sigma(z) = 11\}$:*

```
if (x is even) {
    if (y is odd) {
        z = 42;
    }
}
```

¿La aserción $q = \{\sigma \in \Sigma \mid \sigma(z) = 42\}$ es una sub-aproximación de los estados alcanzables por este código partiendo de la presunción?

Para que la tripleta $[p] \ C \ [ok : q]$ sea válida, la Definición 2.4 exige:

$$q \subseteq \text{post}_{ok}(C)(p)$$

Sin embargo, el estado $\sigma = [z \mapsto 42, x \mapsto 1, y \mapsto 2] \in q$, pero no pertenece a $\text{post}_{ok}(C)(p)$. En este estado, x es impar e y es par, por lo que ninguna ejecución del programa que preserve esos valores puede realizar la asignación $z := 42$. Por lo tanto,

$$q \not\subseteq \text{post}_{ok}(C)(p),$$

y la tripleta no es válida.

Un resultado sub-aproximado válido debe conservar las condiciones que justifican la ejecución de la asignación. Por ejemplo:

$$[z = 11] \ C \ [ok : z = 42 \wedge (x \bmod 2 = 0) \wedge (y \bmod 2 \neq 0)]$$

Razonamiento sobre bucles

El análisis de los bucles iterativos es el mayor cuello de botella en la verificación formal de software. En la HL, para razonar sobre un bucle, es obligatorio encontrar un invariante, es decir, una propiedad matemática que englobe todos los estados posibles y se mantenga verdadera sin importar cuántas veces se ejecute el ciclo. Esta tarea no es trivial y a menudo requiere invención humana o dominios abstractos complejos, lo que dificulta la escalabilidad y automatización de las herramientas.

La IL simplifica este problema, ya que los invariantes de bucle no juegan un rol central en el razonamiento sub-aproximado. Como el objetivo de la IL no es garantizar un comportamiento seguro para todos los caminos de ejecución posibles, sino demostrar que ciertos estados son efectivamente alcanzables desde un estado inicial, puede concentrarse en una cantidad finita de iteraciones.

La herramienta consiste en desenrollar el bucle. Esto le permite a la IL analizar un bucle simplemente simulando la ejecución de su cuerpo un número finito y acotado de veces. Como la

IL es sub-aproximada y permite descartar caminos, encontrar un error luego de desenrollar el bucle un par de veces produce una precondition lógicamente válida, eliminando la necesidad de deducir matemáticamente qué ocurriría en iteraciones infinitas.

Para casos donde se requiere un razonamiento paramétrico, la IL introduce la Regla de la Variante Hacia Atrás (KLEENE (VARIANTE)). Su premisa tiene la forma:

$$\vdash_{\text{IL}} [p(n)] \ C \ [ok : p(n+1)]$$

Por el Lema 2.5, esta tripleta establece que cada estado que satisface $p(n+1)$ posee algún estado predecesor que satisface $p(n)$. Repitiendo este argumento, es posible reconstruir una traza finita hasta un estado inicial descrito por $p(0)$. De esta forma, la regla permite obtener:

$$\vdash_{\text{IL}} [p(0)] \ C^* \ [ok : \exists n.p(n)]$$

La denominación “variante hacia atrás” se refiere a esta reconstrucción de los estados predecesores exigida por la interpretación sub-aproximada de las triplets. En la búsqueda automática de errores, esta técnica permite calcular un resumen general de los estados alcanzables tras múltiples iteraciones exitosas, para luego acoplarlo con un paso final donde se dispara la condición de error.

Soundness de la IL

La validez de las reglas deductivas respecto de la semántica se expresa mediante el Teorema de Soundness.

Teorema 2.7. (*Soundness de la IL*) Para todo comando C , aserciones $p, q \subseteq \Sigma$ y condición de salida ϵ :

$$\vdash_{\text{IL}} [p] \ C \ [\epsilon : q] \implies \models_{\text{IL}} [p] \ C \ [\epsilon : q]$$

El teorema establece que tripleta obtenida a partir de las reglas del sistema es semánticamente válida. Por lo tanto, para cada estado final descrito por q , existe un estado inicial que satisface p y una ejecución real del comando que conecta ambos estados. La demostración mecanizada de este resultado para el lenguaje imperativo utilizado en esta tesina se presenta en el Capítulo 3.

Corolario 2.8. Si

$$\vdash_{\text{IL}} [p] \ C \ [er : q]$$

y $q \neq \emptyset$, entonces existen estados $\sigma_p \in p$ y $\sigma_q \in q$ tales que:

$$(\sigma_p, \sigma_q) \in \llbracket C \rrbracket_{er}$$

El corolario expresa la garantía de ausencia de falsos positivos, donde todo estado contenido en un resultado de error derivado está respaldado por una ejecución real. Si el resultado es satisfacible, entonces existe una ejecución que finaliza en el canal de error.

Relevancia práctica

La relevancia práctica del razonamiento sobre incorrectitud puede observarse en distintos trabajos vinculados con Facebook y Meta. Como antecedente industrial, herramientas como Infer y Zoncolan mostraron que los análisis estáticos composicionales podían integrarse en procesos de desarrollo industriales y aplicarse sobre bases de código de gran tamaño [5, 6]. Aunque estas herramientas son anteriores a la introducción de la IL y no constituyen implementaciones

directas de esta lógica, sus experiencias mostraron tanto la necesidad de escalabilidad como la importancia de producir advertencias útiles para los desarrolladores.

Una evidencia más directa sobre la aplicabilidad de la IL y la ISL es el desarrollo de PulseX [13]. En su evaluación sobre OpenSSL, un proyecto de código abierto que proporciona bibliotecas y herramientas criptográficas para comunicaciones seguras, encontró quince errores previamente desconocidos, que fueron reportados a sus mantenedores y corregidos. Este resultado muestra que la IL no solo puede utilizarse como explicación teórica de analizadores existentes, sino también como fundamento para diseñar nuevas herramientas capaces de detectar errores reales en programas de tamaño considerable.

Sin embargo, para aplicar este tipo de razonamiento a programas con memoria mutable y estructuras enlazadas es necesario incorporar mecanismos de razonamiento local. Estos mecanismos son provistos por la Lógica de Separación, que se presenta a continuación.

2.2. Lógica de Separación (SL)

La HL es efectiva para razonar sobre variables simples, pero su escalabilidad disminuye ante estructuras de datos mutables. El obstáculo principal es el *aliasing*.

La SL introducida por Reynolds [19] y desarrollada por O'Hearn [15], extiende la HL con aserciones que describen la estructura de la memoria. Su objetivo es permitir el razonamiento local, es decir, demostrar el comportamiento de un comando considerando solo el fragmento del *heap* que este utiliza, sin tener que describir el resto de la memoria.

Sean Var el conjunto de variables del programa, Val el conjunto de valores y Loc el conjunto de direcciones de memoria. Un *store* es una función total:

$$\text{Store} = \text{Var} \rightarrow \text{Val},$$

mientras que un *heap* se representa mediante una función parcial finita:

$$\text{Heap} = \text{Loc} \rightarrow \text{Val}$$

El hecho de que el *heap* sea parcial expresa que no todas las direcciones posibles forman parte de la memoria actual descrita. Denotamos por $\text{dom}(h)$ el conjunto de direcciones para las cuales el *heap* h se encuentra definido.

Un estado espacial es un par:

$$\sigma = (s, h) \in \text{Store} \times \text{Heap}$$

Las aserciones de la SL se interpretan sobre estados espaciales $P : \text{Store} \times \text{Heap} \rightarrow \text{Prop}$. Escribimos:

$$(s, h) \models P$$

cuando el estado compuesto por el *store* s y el *heap* h satisface la aserción P .

Como mencionamos antes, el principal problema que busca controlar la SL es el *aliasing*.

Definición 2.9 (Aliasing). Dos expresiones de puntero e_1 y e_2 presentan *aliasing* en un *store* s cuando ambas evalúan a la misma dirección de memoria:

$$\llbracket e_1 \rrbracket_s = \llbracket e_2 \rrbracket_s$$

En este tipo de programas, una modificación realizada a través de un puntero puede alterar el valor al que hace referencia otro puntero diferente. Entonces, para poder probar la correctitud de un programa con *aliasing* mediante la lógica tradicional, el verificador debe añadir restricciones globales complejas que garanticen que ninguna operación de mutación afectará a otras referencias. Este es un proceso que, para programas de tamaño moderado, se le suma mucha dificultad y no escala.

La conjunción separadora y aserciones espaciales

Antes de definir las aserciones espaciales, es necesario especificar cuándo dos fragmentos de memoria son independientes.

Definición 2.10 (Disjunción de *heaps*). Dos *heaps* h_1 y h_2 son disjuntos, lo que denotamos mediante $h_1 \perp h_2$, si sus dominios no comparten ninguna dirección:

$$\text{dom}(h_1) \cap \text{dom}(h_2) = \emptyset$$

Cuando $h_1 \perp h_2$, puede construirse su unión disjunta:

$$h_1 \uplus h_2$$

Esta operación contiene todas las celdas de ambos *heaps*:

$$\text{dom}(h_1 \uplus h_2) = \text{dom}(h_1) \cup \text{dom}(h_2)$$

Cuando sus dominios son disjuntos, ninguna dirección recibe dos valores diferentes y la unión se encuentra bien definida.

La SL incorpora aserciones específicas para describir fragmentos de memoria. Entre las más importantes se encuentran **emp**, **points-to** y la conjunción separadora.

Definición 2.11 (*Heap* vacío). La aserción **emp** se satisface cuando el *heap* no contiene ninguna celda:

$$(s, h) \models \text{emp} \iff \text{dom}(h) = \emptyset$$

La aserción **emp** no afirma que toda la memoria física del programa se encuentra vacía, sino que el fragmento del *heap* descrito por la aserción es vacío. Esta distinción resulta útil porque las aserciones de SL describen recursos locales y pueden combinarse mediante la conjunción separadora.

Definición 2.12 (Points-to). La aserción

$$e_1 \mapsto e_2$$

se satisface cuando el *heap* contiene exactamente una celda, cuya dirección es el valor de e_1 y cuyo contenido es el valor de e_2 . Formalmente:

$$(s, h) \models e_1 \mapsto e_2 \iff \exists \ell \in \text{Loc}. \ell = \llbracket e_1 \rrbracket_s \wedge \text{dom}(h) = \{\ell\} \wedge h(\ell) = \llbracket e_2 \rrbracket_s$$

Por ejemplo, $x \mapsto 3$ describe un *heap* formado exactamente por una celda ubicada en la dirección almacenada en x , cuyo contenido es 3. Esta aserción no describe un *heap* arbitrariamente grande que contiene esa celda, sino un fragmento compuesto únicamente por ella. Para describir otras celdas independientes deben agregarse nuevas aserciones mediante la conjunción separadora.

Definición 2.13 (Conjunción separadora). Sean P y Q dos aserciones espaciales. El estado (s, h) satisface la conjunción separadora

$$P * Q$$

si el *heap* h puede dividirse en dos fragmentos disjuntos h_1 y h_2 , de modo que P se satisface

sobre h_1 y Q se satisface sobre h_2 . Formalmente:

$$(s, h) \models P * Q \iff \exists h_1, h_2 \in \text{Heaps}. h_1 \perp h_2 \wedge h = h_1 \uplus h_2 \wedge (s, h_1) \models P \wedge (s, h_2) \models Q$$

El contraste entre la conjunción clásica ($\wedge$) y la separadora ($*$) es lo que le otorga a la SL el poder de describir patrones de *aliasing* o separación de forma concisa. Para ilustrar esto visualmente, consideramos la aserción $x \mapsto 3, y$, la cual indica que x apunta a un bloque de dos celdas adyacentes que contienen los valores 3 e y respectivamente, y la aserción $y \mapsto 3, x$. Ambas aserciones se representan individualmente en la Figura 2.2.



Figura 2.2: Representación individual de las aserciones espaciales $x \mapsto 3, y$ e $y \mapsto 3, x$.

Si analizamos la aserción $(x \mapsto 3, y) \wedge (y \mapsto 3, x)$, la conjunción clásica dicta que ambos predicados se cumplen sobre la misma memoria exacta. Físicamente, esto es posible si x e y son exactamente el mismo puntero, como se muestra en la Figura 2.3.

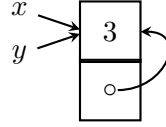


Figura 2.3: Conjunción clásica.

Por el contrario, si utilizamos la conjunción separadora $(x \mapsto 3, y) * (y \mapsto 3, x)$, estamos forzando lógicamente a que las memorias descritas sean disjuntas. Como muestra la Figura 2.4, el resultado es una estructura en la que existen dos bloques de memoria separados, donde cada uno guarda un puntero hacia el otro.

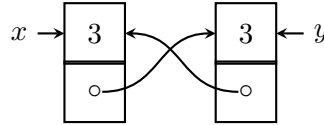


Figura 2.4: Conjunción separadora.

Luego, la prohibición de *aliasing* indeseado queda construida a nivel lógico.

Lema 2.14 (Propiedades de la conjunción separadora). *Para todas las aserciones espaciales P , Q y R :*

1. $P * Q \iff Q * P$
2. $(P * Q) * R \iff P * (Q * R)$
3. $P * \text{emp} \iff P$

El lenguaje imperativo extendido

Para poder operar sobre este nuevo modelo de memoria, la SL extiende el lenguaje de programación imperativo clásico con cuatro comandos fundamentales para la manipulación de estructuras de datos mutables:

1. **Asignación (allocation):**

$$v := \text{cons}(e_1, \dots, e_n)$$

reserva n celdas consecutivas del heap y guarda la dirección base en la variable v .

2. **Lectura (lookup):**

$$v := [e]$$

lee el valor guardado en la dirección de memoria e del heap y lo guarda en el store en la variable v .

3. **Mutación (mutation):**

$$[e] := e'$$

sobrescribe el contenido del heap en la dirección e con el nuevo valor e' .

4. **Liberación (deallocation):**

$$\text{dispose } e$$

invalida y libera la dirección de memoria e , eliminándola del dominio del heap activo.

Razonamiento local y la Regla del Marco

La capacidad de describir porciones disjuntas de memoria da lugar al principio más importante de esta teoría en lo que respecta a la especificación de programas, el razonamiento local. Según Reynolds [19], para entender cómo funciona un programa, debe ser posible que el razonamiento y la especificación se limiten a las celdas de memoria que el programa realmente accede. El valor de cualquier otra celda permanecerá automáticamente sin cambios.

Ejemplo 2.2. *La tripleta*

$$\{x \mapsto v\} \quad [x] := w \quad \{x \mapsto w\}$$

describe localmente una escritura en memoria. La precondition contiene solo la celda a la que se accede, y la postcondition describe su nuevo valor.

Supongamos ahora que el programa se ejecuta en un heap más grande que contiene además una celda independiente $y \mapsto u$. La escritura mediante x no debería modificar esta segunda celda. Por lo tanto, debería ser posible obtener:

$$\{x \mapsto v * y \mapsto u\} \quad [x] := w \quad \{x \mapsto w * y \mapsto u\}$$

Este principio se materializa mediante la Regla del Marco (*Frame Rule*), que permite razonar exclusivamente sobre la parte de la memoria que un comando realmente modifica, ignorando el resto del heap.

Sean P , Q y R aserciones espaciales y sea C un comando, definimos la Regla del Marco como:

$$\frac{\vdash_{\text{SL}} \{P\} \quad C \quad \{Q\} \quad \text{mod}(C) \cap \text{fv}(R) = \emptyset}{\vdash_{\text{SL}} \{P * R\} \quad C \quad \{Q * R\}}$$

En la condición lateral tenemos $\text{mod}(C)$, el conjunto de variables del *store* que pueden ser modificadas por C , y $\text{fv}(R)$, el conjunto de variables libres de la aserción R . Esta condición garantiza que C no modifique ninguna variable de las que depende R , de modo que este recurso siga describiendo el mismo fragmento de memoria después de ejecutar C .

Esta regla formaliza la intuición de la localidad. Si un programa funciona correctamente sobre un fragmento del *heap*, entonces seguirá funcionando de la misma manera si agregamos un recurso ajeno R que no interfiere en la ejecución. La SL garantiza que el recurso R quedará inalterado, permitiendo razonar sobre sistemas complejos a través de componentes más simples. Su fundamento semántico es la propiedad de localidad, que establece que una ejecución sobre un fragmento de *heap* puede extenderse con memoria disjunta que permanece inalterada.

Lema 2.15 (Localidad operacional). Sean C un comando, ϵ una condición de salida, h y h' los heaps inicial y final de una ejecución y, h_f un fragmento adicional. Si

$$((s, h), (s', h')) \in \llbracket C \rrbracket_\epsilon \wedge h \perp h_f \wedge h' \perp h_f$$

entonces

$$((s, h \uplus h_f), (s', h' \uplus h_f)) \in \llbracket C \rrbracket_\epsilon$$

El Lema 2.15 establece que el mismo fragmento h_f puede agregarse a los estados inicial y final sin alterar la ejecución local. El comando transforma el *heap* h en h' , mientras que h_f no se modifica.

La SL proporciona las herramientas necesarias para razonar localmente sobre memoria mutable. Sin embargo, combinar su modelo clásico de *heap* con la sub-aproximación de la IL presenta una dificultad. En la semántica tradicional, la liberación de una celda elimina el recurso correspondiente del *heap*. Al aplicar la Regla del Marco en un sistema sub-aproximado, esta desaparición puede transformar una presunción inconsistente en un resultado satisfacible, violando la garantía de alcanzabilidad establecida por el Teorema de Soundness 2.7.

2.3. Lógica de Incorrectitud de Separación (ISL)

La SL proporciona técnicas novedosas para el razonamiento local, permitiendo que la verificación formal escale a bases de código muy grandes. Por su lado, la IL permite demostrar la alcanzabilidad de estados concretos mediante sub-aproximación. La Lógica de Incorrectitud de Separación, introducida por Raad et al. [17] fusiona el razonamiento local proporcionado por la conjunción separadora y la Regla del Marco de la SL con las tripletas de sub-aproximación de la IL.

Una tripleta de la ISL conserva la forma presentada para la IL: $[p] \ C \ [\epsilon : q]$, pero ahora p y q son aserciones espaciales sobre estados compuestos por un *store* y un *heap*. Su interpretación semántica sigue siendo sub-aproximada:

$$\models_{\text{ISL}} [p] \ C \ [\epsilon : q] \iff \forall \sigma_q \in q. \exists \sigma_p \in p. (\sigma_p, \sigma_q) \in \llbracket C \rrbracket_\epsilon$$

Por lo tanto, todo estado descrito por q debe estar respaldado por una ejecución real de C desde algún estado que satisface p .

Heaps negativos

Intentar construir la ISL combinando linealmente la semántica de la SL tradicional y la IL produce un sistema inválido. En el modelo clásico de la SL, la operación de liberación (**free**) remueve la locación del *heap*, haciendo que el estado vuelva a ser una porción vacía. Si intentamos aplicar la Regla de Marco bajo el pensamiento de sub-aproximación, el sistema colapsa porque nos permite deducir un estado final válido partiendo de una precondition imposible.

En el modelo clásico de la SL, la regla local para liberar una celda puede expresarse como:

$$[x \mapsto -] \ \text{free}(x) \ [ok : \text{emp}]$$

La celda desaparece del *heap* después de ser liberada.

Supongamos que aplicamos la Regla del Marco utilizando como marco otra celda descrita por $x \mapsto -$:

$$\frac{\vdash_{\text{ISL}} [x \mapsto -] \ \text{free}(x) \ [ok : \text{emp}]}{\vdash_{\text{ISL}} [x \mapsto - * x \mapsto -] \ \text{free}(x) \ [ok : \text{emp} * x \mapsto -]}$$

La presunción $x \mapsto - * x \mapsto -$ es inconsistente porque exige que dos fragmentos disjuntos posean la misma dirección. No existe ningún estado que la satisfaga. Sin embargo, el resultado $\text{emp} * x \mapsto -$ es satisfacible. Esto produciría una tripleta que pretende alcanzar un estado real a partir de un conjunto vacío de estados iniciales, contradiciendo la interpretación sub-aproximada.

El problema no pertenece a la IL en sí misma, sino a la combinación directa de la sub-aproximación, la semántica clásica de liberación y la Regla del Marco. El modelo clásico no permite distinguir entre una dirección sobre la cual no se posee información y una dirección que se sabe que fue liberada. Esta distinción es la que introducen los *heaps* negativos.

Definición 2.16 (*Heap extendido*). Sean Loc el conjunto de ubicaciones de memoria y Val el conjunto de valores. Un *heap* de la ISL es una función parcial finita:

$$h : \text{Loc} \rightarrow (\text{Val} \cup \{\perp\})$$

, donde:

- $h(\ell) = v$ indica que ℓ se encuentra asignada y contiene v ;
- $h(\ell) = \perp$ indica que ℓ fue liberada y todavía no fue reasignada;
- $\ell \notin \text{dom}(h)$ indica que la aserción no posee información sobre esa ubicación.

El símbolo $\perp$ no representa una dirección inexistente. Representa una dirección conocida cuya celda se encuentra actualmente liberada.

Definición 2.17 (*Heap negativo*). La aserción:

$$e \not\vdash$$

se satisface cuando el *heap* está formado por una celda liberada ubicada en la dirección indicada por e :

$$(s, h) \models e \not\vdash \iff \exists \ell \in \text{Loc}. \llbracket e \rrbracket_s = \ell \wedge \text{dom}(h) = \{\ell\} \wedge h(\ell) = \perp$$

Por lo tanto, $e \not\vdash$ expresa que la dirección almacenada en e :

1. es una ubicación conocida;
2. fue liberada;
3. todavía no fue reasignada.

Esta aserción se puede leer como que la ubicación de memoria indicada por e se encuentra invalidada.

Si volvemos a aplicar la Regla del Marco, el *heap* negativo se arrastra hasta el final y contamina el resultado. Luego, la regla local de liberación se convierte en:

$$\vdash_{\text{ISL}} [x \mapsto -] \text{ free}(x) [ok : x \not\vdash]$$

Al aplicar otra vez el marco $x \mapsto -$ obtenemos:

$$\vdash_{\text{ISL}} [x \mapsto - * x \mapsto -] \text{ free}(x) [ok : x \not\vdash * x \mapsto -]$$

La presunción sigue siendo inconsistente, pero ahora el resultado también, ya que la misma dirección pertenece a dos *heaps* disjuntos, estando liberada en uno y asignada en el otro. Por lo tanto, la conclusión es semánticamente válida:

$$\emptyset \subseteq \text{post}_{ok}(\text{free}(x))(\emptyset)$$

Los *heaps* negativos impiden que la Regla del Marco transforme una presunción inconsistente en un resultado satisfacible.

Representamos el efecto del *heap* negativo en la Tabla 2.1:

Presunción inconsistente	Comando	Resultado satisfacible
$[x \mapsto - * x \mapsto -]$	<code>free(x)</code>	$[\text{emp} * x \mapsto -]$
Presunción inconsistente	Comando	Resultado inconsistente
$[x \mapsto - * x \mapsto -]$	<code>free(x)</code>	$[x \not\mapsto * x \mapsto -]$

Tabla 2.1: Preservación de la inconsistencia mediante *heaps* negativos.

La conservación de las celdas liberadas produce una propiedad fundamental del modelo de la ISL.

Lema 2.18 (Monotonía del *heap*). *Para todo comando C , canal de salida ϵ y transición $((s, h), (s', h')) \in \llbracket C \rrbracket_\epsilon$, se cumple:*

$$\text{dom}(h) \subseteq \text{dom}(h')$$

Esto quiere decir que una ubicación conocida al comienzo de una ejecución sigue siendo conocida cuando esta finaliza.

La monotonía puede observarse en las operaciones básicas:

- una asignación sobre una dirección nueva amplía el dominio;
- una lectura o escritura conserva el dominio;
- una liberación transforma v en $\perp$, pero no elimina la dirección;
- una reasignación transforma $\perp$ a un valor, pero tampoco modifica el dominio

Por lo tanto, el dominio puede crecer o permanecer igual, pero nunca disminuir.

Reglas de Inferencia

Las reglas estructurales de la IL se conservan en la ISL. La diferencia se encuentra en las que manipulan la memoria dinámica y en la incorporación de la Regla del Marco. Estas nuevas reglas se presentan en la Figura 2.5.

La regla de asignación de memoria `ALLOC1` representa la reserva de una dirección nueva sobre la cual el *heap* local no poseía información. La dirección debe ser fresca respecto del fragmento de memoria considerado. De esta manera, la reserva amplía el dominio conocido del *heap*. Por otro lado, la regla `ALLOC2` formaliza la capacidad de reasignar una celda de memoria conocida que estaba liberada. A diferencia de la anterior, esta regla no incorpora una nueva dirección al dominio.

`FREE` representa la liberación exitosa, toma una celda válida y activa en la memoria y muta su estado lógico hacia un *heap* negativo. Por otro lado, `FREEER` y `REENULL` representan el caso de error. El primero intercepta la violación de tipo *double free*; si se intenta liberar un puntero que ya está marcado como inválido, el sistema aborta hacia el canal de error. El segundo caso intercepta el fallo provocado por el puntero nulo.

$$\begin{array}{c}
\text{ALLOC1} \\
\vdash_{\text{ISL}} [x = x'] \quad x := \text{alloc}() \quad [ok : x \mapsto -] \\
\\
\begin{array}{cc}
\text{ALLOC2} & \text{FREE} \\
\vdash_{\text{ISL}} [x = x' * y \nrightarrow] \quad x := \text{alloc}() \quad [ok : x = y * y \mapsto -] & \vdash_{\text{ISL}} [x \mapsto e] \quad \text{free}(x) \quad [ok : x \nrightarrow]
\end{array} \\
\\
\begin{array}{cc}
\text{FREEER} & \text{FREENULL} \\
\vdash_{\text{ISL}} [x \nrightarrow] \quad \text{free}(x) \quad [er : x \nrightarrow] & \vdash_{\text{ISL}} [x = \text{null}] \quad \text{free}(x) \quad [er : x = \text{null}]
\end{array} \\
\\
\begin{array}{cc}
\text{LOAD} & \text{LOADER} \\
\vdash_{\text{ISL}} [x = x' * y \mapsto e] \quad x := [y] \quad [ok : x = e[x'/x] * y \mapsto e[x'/x]] & \vdash_{\text{ISL}} [y \nrightarrow] \quad x := [y] \quad [er : y \nrightarrow]
\end{array} \\
\\
\begin{array}{cc}
\text{LOADNULL} & \text{STORE} \\
\vdash_{\text{ISL}} [y = \text{null}] \quad x := [y] \quad [er : y = \text{null}] & \vdash_{\text{ISL}} [x \mapsto e] \quad [x] := y \quad [ok : x \mapsto y]
\end{array} \\
\\
\begin{array}{cc}
\text{STOREER} & \text{STORENULL} \\
\vdash_{\text{ISL}} [x \nrightarrow] \quad [x] := y \quad [er : x \nrightarrow] & \vdash_{\text{ISL}} [x = \text{null}] \quad [x] := y \quad [er : x = \text{null}]
\end{array}
\end{array}$$

Figura 2.5: Reglas de inferencia de la ISL. Adaptadas de [17].

Las reglas LOAD rigen la lectura del *heap*. El caso de éxito recupera hacia adelante el valor indexado en el *store* mediante una variable x' . La regla LOADER detiene el flujo si el programa intenta realizar una lectura sobre un puntero previamente liberado (*use after free*). Por último, LOADNULL detiene el flujo cuando el puntero que se quiere leer es nulo.

Finalmente, las reglas de STORE controlan la mutación del *heap*. En particular, el caso de éxito actualiza el valor de la celda de forma local. Análogamente a los casos anteriores, STOREER interrumpe el flujo si la celda apuntada por x es inválida, mientras que STORENULL hace lo mismo para el caso donde se trate de un puntero nulo.

Con respecto a la Regla del Marco, en la ISL se formaliza de la siguiente manera:

$$\frac{\text{FRAME} \quad \vdash_{\text{ISL}} [p] \quad C \quad [\epsilon : q] \quad \text{mod}(C) \cap \text{fv}(r) = \emptyset}{\vdash_{\text{ISL}} [p * r] \quad C \quad [\epsilon : q * r]} \quad (2.1)$$

A diferencia de la SL tradicional, donde la regla del marco puede aplicarse solo en ejecuciones seguras, en la ISL la regla se aplica de forma idéntica tanto en el canal de éxito como en el de error. Gracias a que los *heaps* negativos retienen el espacio de las celdas liberadas, un error local detectado en una parte pequeña de la memoria se propaga al *heap* global a través del marco r , garantizando que el *bug* encontrado es real y escalable a sistemas mayores.

Soundness de la ISL

Al igual que para la IL, expresamos la validez de las reglas deductivas respecto a la semántica con el Teorema de Soundness.

Teorema 2.19 (Soundness de la ISL). *Para todo comando C , aserciones espaciales p y q , y canal de salida ϵ :*

$$\vdash_{\text{ISL}} [p] \quad C \quad [\epsilon : q] \implies \models_{\text{ISL}} [p] \quad C \quad [\epsilon : q]$$

Al igual que en la IL, el teorema establece que las reglas de inferencia de la ISL, incluida la Regla del Marco, no permiten introducir estados finales que no estén respaldados por una

ejecución real. En particular, si se deriva $\vdash_{\text{ISL}} [p] \ C \ [er : q]$, entonces cada estado de error contenido en q es alcanzable desde algún estado que satisface p .

Para afirmar además que el programa tiene una ejecución errónea, el resultado q debe ser satisfacible. Si $q = \emptyset$, la tripleta es válida de manera trivial, pero no demuestra la existencia de ningún error.

Corolario 2.20. *Si*

$$\vdash_{\text{ISL}} [p] \ C \ [er : q]$$

y $q \neq \emptyset$, entonces existen estados $\sigma_p \in p$ y $\sigma_q \in q$ tales que:

$$(\sigma_p, \sigma_q) \in \llbracket C \rrbracket_{er}$$

La conservación de esta propiedad al incorporar la Regla del Marco depende de que los recursos utilizados permanezcan sin modificaciones durante la ejecución. La propiedad de localidad y el Teorema de la Huella, presentados a continuación, explican semánticamente por qué este razonamiento es válido.

El Teorema de la Huella (*Footprint Theorem*)

En el Lema 2.15 se presentó la propiedad de localidad, según la cual una ejecución sobre un fragmento local del *heap* puede extenderse con un fragmento disjunto que permanece sin modificaciones. Esta propiedad explica la dirección local a global del razonamiento por marcos.

En la ISL, el Teorema de la Huella proporciona una caracterización más completa. En términos generales, además de permitir la extensión de ejecuciones locales, establece que toda ejecución global puede descomponerse en una ejecución sobre una porción mínima de memoria y un marco adicional que no participa de la ejecución.

Para expresar la extensión de un estado con memoria adicional, consideremos un estado espacial $\sigma = (s, h)$ y un *heap* h_f tal que $h \perp h_f$. Definimos:

$$\sigma \bullet h_f = (s, h \uplus h_f)$$

Esta operación agrega al estado el fragmento disjunto h_f , sin modificar el *store*.

Definición 2.21 (Extensión por marcos). Sea $R \subseteq \Sigma \times \Sigma$ una relación entre estados. Su extensión por marcos se define como:

$$\text{frame}(R) = \{(\sigma_p \bullet h_f, \sigma_q \bullet h_f) \mid (\sigma_p, \sigma_q) \in R \wedge h(\sigma_p) \perp h_f \wedge h(\sigma_q) \perp h_f\}$$

La Definición 2.21 agrega el mismo fragmento h_f al estado inicial y al estado final de una transición. Las condiciones de disjunción garantizan que este fragmento no se superpone con la memoria local utilizada por la ejecución.

Dos transiciones pueden compararse según la cantidad de memoria adicional que contienen. Decimos que una transición (σ_p, σ_q) es menor o igual que (σ'_p, σ'_q) respecto de la extensión por marcos, y escribimos $(\sigma_p, \sigma_q) \preceq (\sigma'_p, \sigma'_q)$ si existe un *heap* h_f tal que:

$$\sigma'_p = \sigma_p \bullet h_f \wedge \sigma'_q = \sigma_q \bullet h_f$$

Con esto podemos definir la huella de un comando o *footprint*.

Definición 2.22 (Huellas). La relación

$$\text{foot}(C)_\epsilon$$

está formada por las transiciones de $\llbracket C \rrbracket_\epsilon$ que son mínimas respecto del orden $\preceq$.

Una transición perteneciente a $\text{foot}(C)_\epsilon$ describe los recursos de memoria necesarios para una ejecución particular del comando. No necesariamente existe una única huella para cada comando. Un comando con distintas ramas puede tener varias ejecuciones mínimas con requerimientos de memoria distintos. Por ejemplo, una ejecución exitosa de la instrucción $[x] := y$ requiere localmente una celda activa en la dirección almacenada en x , pero no necesita describir las demás celdas de *heap*. El resto de la memoria puede incorporarse con un marco.

Teorema 2.23 (Teorema de la Huella). *Para todo comando C y toda condición de salida $\epsilon \in \{ok, er\}$,*

$$\llbracket C \rrbracket_\epsilon = \text{frame}(\text{foot}(C)_\epsilon)$$

La igualdad del Teorema 2.23 junta dos propiedades diferentes. En primer lugar, $\text{frame}(\text{foot}(C)_\epsilon) \subseteq \llbracket C \rrbracket_\epsilon$. Esta inclusión expresa la dirección local a global. Toda transición mínima sigue siendo una ejecución válida cuando se le agrega un marco compatible. Esta dirección es una aplicación de la propiedad de localidad presentada en el Lema 2.15, generalizada en la ISL a los canales de terminación errónea y normal.

En segundo lugar, $\llbracket C \rrbracket_\epsilon \subseteq \text{frame}(\text{foot}(C)_\epsilon)$. Esta inclusión expresa la dirección global a local. Establece que toda ejecución global puede descomponerse en una transición mínima y un marco que permanece sin modificaciones.

En el Capítulo 4 se mecanizan ambas direcciones del Teorema de la Huella 2.23. La dirección local a global se obtiene mediante la extensión operacional por marco, mientras que la dirección global a local demuestra que toda ejecución admite una descomposición en una transición mínima y un marco disjunto.

Begin-Anywhere

La combinación de razonamiento local, monotonía del *heap* y sub-aproximación permite construir análisis de ejecución simbólica capaces de comenzar desde un fragmento de código sin conocer previamente todo el estado global. Esta estrategia se conoce como *Begin-Anywhere*. En las herramientas de verificación clásicas, siempre se debe empezar desde un punto de entrada raíz (como la función `main`) para tener todo el contexto del *heap*. Sin este contexto, el analizador no sabría qué hay en la memoria al llegar a una función intermedia y fallaría.

En la ISL, el análisis puede empezar en cualquier procedimiento de forma aislada gracias a la técnica de bi-abducción adaptada a la sub-aproximación [4]. Podemos definir la bi-abducción como la capacidad del analizador para sintetizar lógicamente las precondiciones faltantes sobre la marcha. Si al analizar una instrucción falta un recurso en el estado actual, la herramienta infiere que ese recurso debió existir en el estado inicial para que la ejecución fuera posible. En esta tesina no se implementa dicho procedimiento automático, las presunciones, los caminos y las derivaciones se construyen de forma explícita, y F^* verifica su validez.

Al sintetizar el contexto necesario paso a paso enfocándose estrictamente en la huella mínima del comando, la herramienta puede descubrir vulnerabilidades de forma modular. Debido a que la ISL trabaja bajo el paradigma de la sub-aproximación, esta técnica es muy potente. Cada error que el sistema encuentre partiendo de estas precondiciones sintetizadas es un escenario real y alcanzable.

2.4. El lenguaje F^*

Para garantizar el rigor matemático de los sistemas deductivos complejos, es una práctica estándar mecanizar su semántica y teoremas en un asistente de pruebas. En este trabajo, la

formalización completa se desarrolla utilizando F^* , un lenguaje de programación funcional con tipos dependientes que también funciona como asistente de pruebas y verificador de programas [20]. Este lenguaje está integrado con el solucionador SMT Z3. Su paradigma se denomina programación orientada a pruebas (*proof-oriented programming*), ya que los programas, sus especificaciones y sus demostraciones se desarrollan dentro de un mismo lenguaje.

Además de utilizarse como asistente de pruebas, F^* puede emplearse para desarrollar programas ejecutables. Los fragmentos relevantes pueden extraerse a lenguajes como OCaml y F#, y el ecosistema también provee mecanismos de compilación a C y WebAssembly. Esta capacidad de extracción no se utiliza directamente en la formalización presentada, cuyo contenido es principalmente lógico y fantasma, pero forma parte del carácter híbrido de F^* como lenguaje de programación y verificación.

En esta tesina, F^* no se utiliza para implementar un analizador automático ni para producir código ejecutable. Su función es mecanizar la semántica operacional de los lenguajes considerados, representar los sistemas deductivos de la IL y la ISL, y comprobar sus propiedades metateóricas. En particular, permite expresar las reglas de inferencia como tipos inductivos y demostrar *soundness* mediante funciones recursivas sobre las derivaciones.

A continuación se introducen las características del lenguaje necesarias para comprender la formalización desarrollada en los siguientes capítulos.

Tipos dependientes y refinamientos

En F^* , los tipos pueden depender de valores. La manifestación más práctica de esto son los tipos de refinamiento. Un tipo de refinamiento permite expresar propiedades de los datos directamente en sus tipos, de modo que su cumplimiento sea comprobado estáticamente.

Dado un tipo t y un predicado P , la expresión $x : t \{ P \ x \}$ representa los valores x de tipo t que además satisfacen la propiedad $P \ x$. Por ejemplo, en la biblioteca estándar de F^* , el tipo de los números naturales no es un tipo primitivo aislado, sino el tipo de los enteros refinado por una condición booleana:

```
let nat = x : int { x >= 0 }
```

Cualquier función que requiera un argumento del tipo `nat` fuerza al verificador estático a demostrar matemáticamente que el valor provisto satisface la restricción $x \geq 0$ antes de compilar. De este modo, propiedades que en otros lenguajes se verificarían durante la ejecución se convierten en obligaciones estáticas.

Esta dependencia entre valores y tipos permite que la firma de una función actúe como su propia especificación matemática. Por ejemplo, la función de incremento puede definirse garantizando estáticamente que el valor de retorno y será exactamente igual al argumento x más uno:

```
val incr : x : int -> y : int { y = x + 1 }
let incr x = x + 1
```

En la formalización desarrollada, las aserciones lógicas se representan como predicados sobre estados:

```
type cond = state -> prop
```

A su vez, los refinamientos permiten exigir que los estados recibidos o producidos por una función satisfagan una aserción determinada. Por ejemplo, el siguiente parámetro:

```
s : state { post s }
```

representa un estado s acompañado por la garantía estática de que satisface la postcondición `post`. Esta forma aparece en el enunciado mecanizado del Teorema de Soundness, donde la prueba recibe un estado final que satisface el resultado de una tripleta y debe construir un estado inicial que satisfaga su presunción.

Tipos inductivos indexados y Curry-Howard

F* hace un uso extensivo de los tipos inductivos para definir estructuras de datos y proposiciones lógicas. Una de las características más potentes del lenguaje es la capacidad de crear tipos inductivos indexados, los cuales permiten capturar invariantes complejos directamente en la forma de la estructura.

La formalización se apoya en la correspondencia Curry-Howard [11], donde las proposiciones se interpretan como tipos y las demostraciones como valores que habitan esos tipos. La ventaja de esto es que F* obliga a considerar todos los constructores al demostrar una metapropiedad por inducción estructural. En particular, la función de *soundness* analiza la forma de una derivación y construye, para cada regla, el estado inicial y la traza operacional que justifican el estado final.

Un ejemplo clásico en la programación de tipos dependientes es el tipo de los vectores, donde la longitud forma parte del propio tipo:

```
type vec (a : Type) : nat -> Type =  
  | Nil : vec a 0  
  | Cons : #n : nat -> hd : a -> tl : vec a n -> vec a (n + 1)
```

El constructor `Nil` solo produce vectores de longitud cero, mientras que `Cons` recibe un vector de longitud `n` y produce uno de longitud `n + 1`. Por lo tanto, la longitud no es una propiedad externa que deba verificarse luego, sino un invariante impuesto por el sistema de tipos. Esta información permite definir funciones de acceso cuyos índices estén refinados por la longitud del vector, excluyendo estáticamente los accesos fuera de rango.

Esta característica se utiliza en la tesina para representar tanto la semántica operacional como los sistemas deductivos. La relación `runsto : stmt -> state -> state -> Type0` es un tipo indexado por un comando y dos estados. Un habitante de `runsto c s0 s1` constituye una derivación que certifica que el comando `c` puede ejecutarse desde un estado `s0` hasta otro estado `s1`. Cada constructor de `runsto` corresponde a una regla de la semántica operacional.

De manera similar, el tipo `il_triple : cond -> stmt -> cond -> Type` representa las derivaciones admitidas por el sistema deductivo de la IL. Las admitidas por la ISL tienen una estructura similar. Un valor de tipo `il_triple pre c post` representa la derivación de la tripleta de incorrectitud correspondiente. Cada regla lógica se codifica como un constructor del tipo.

Pruebas fantasma y automatizaciones

La mayor parte de la formalización tiene contenido lógico y no está destinada a ejecutarse. F* permite distinguir este tipo de definiciones mediante cómputos fantasma, que son comprobados por el verificador pero eliminados durante la extracción de código.

El efecto `GTot` describe una función total y fantasma. Por ejemplo:

```
let eval_expr (s : state) (e : expr) : GTot int =
```

indica que la función siempre termina y que su resultado se utiliza solo dentro de especificaciones o demostraciones.

Para enunciar y demostrar teoremas abstractos, F* proporciona el efecto `Lemma`. Un lema toma ciertas premisas como argumentos mediante una cláusula `requires` y garantiza lógicamente una conclusión mediante la cláusula `ensures`. Su resultado computacional es irrelevante, lo importante es que su cuerpo constituya una demostración aceptada por el comprobador. Por ejemplo, una firma de la forma:

```
val lemma :  
  x : t ->  
  Lemma  
    (requires P x)  
    (ensures Q x)
```

establece que, siempre que se cumpla la premisa $P \ x$, el lema garantiza la conclusión $Q \ x$.

Otra construcción utilizada en la formalización es **squash**. Dada una proposición P , el tipo **squash** P conserva únicamente la información de que P es verdadera, sin conservar una estructura computacional de su demostración. Por eso se habla de una prueba “aplastada” o irrelevante desde el punto de vista computacional. Esta construcción resulta adecuada para condiciones laterales como:

```
squash (forall s. pre s ==> pre' s)
```

que es una condición lateral de la Regla de Consecuencia.

En este caso, la formalización solo necesita establecer que la implicación es válida. No necesita inspeccionar posteriormente cómo fue demostrada. El uso de **squash** también facilita que estas obligaciones sean tratadas por el *solver* SMT Z3. El *solver* puede resolver automáticamente muchas obligaciones de primer orden, como implicaciones, igualdades, restricciones aritméticas e inclusiones entre predicados.

Sin embargo, esta automatización no elimina la necesidad de construir las demostraciones. El usuario debe proporcionar la estructura principal de los argumentos, especialmente cuando requieren inducción o la construcción de testigos existenciales. Cuando el *solver* no puede resolver una obligación automáticamente, es necesario agregar lemas auxiliares, revelar definiciones o proporcionar pasos intermedios que orienten la prueba.

Además, las funciones recursivas tienen que demostrar su terminación. Cuando esto no resulta evidente de manera automática, F^* permite indicar una medida decreciente mediante la cláusula **decreases**. En las demostraciones de *soundness*, la medida utilizada es la propia derivación, ya que cada llamada recursiva se realiza sobre una sub-derivación estrictamente menor.

Estas características determinan la estrategia de mecanización utilizada en los capítulos siguientes. Las aserciones se representan como predicados sobre estados; las semánticas operacionales y los sistemas deductivos, como tipos inductivos indexados; y sus propiedades metateóricas, como funciones recursivas y lemas fantasma. La integración con Z3 permite automatizar las obligaciones lógicas locales, mientras que la estructura global de cada demostración permanece explícita en el código de F^* .

Capítulo 3

Formalización de IL

“When you have eliminated the impossible, whatever remains, however improbable, must be the truth.”

— Arthur Conan Doyle

En el capítulo anterior se establecieron los fundamentos teóricos de la IL. En este capítulo se presenta la mecanización de este sistema deductivo utilizando el asistente de pruebas F*.

Para lograrlo, se aplica el paradigma de programación orientada a pruebas. En primer lugar, se modela la sintaxis y semántica del lenguaje imperativo subyacente. Luego, se codifican los axiomas de la IL a través del isomorfismo de Curry-Howard. Por último, se demuestra su solidez (*soundness*) respecto a la semántica operacional.

3.1. Lenguaje y semántica operacional

Para poder razonar sobre el comportamiento de un programa, el primer paso es definir formalmente su sintaxis y semántica. En esta formalización inicial de la IL trabajaremos sobre un lenguaje imperativo simplificado.

El conjunto abstracto de estados Σ , utilizado en la Sección 2.1, se instancia en esta primera formalización mediante un *store* y un modo de terminación. Las variables se representan mediante cadenas y sus valores mediante números naturales:

```
type var = string
type value = nat
type store = var -> value
```

Un *store* es una función total que asigna un valor a cada variable. La actualización de una variable se expresa funcionalmente mediante la siguiente función:

```
let override (#a : eqtype) (#b : Type)
  (f : a -> b) (x : a) (y : b) : a -> b =
  fun z -> if z = x then y else f z
```

La expresión `override f x` y produce una nueva función que devuelve `y` para el argumento `x` y conserva el comportamiento de `f` para todos los demás argumentos.

El lenguaje considerado distingue entre ejecuciones que continúan normalmente y ejecuciones que han alcanzado un error. Para representar esta información se definen los siguientes tipos:

```
type term_mode =
  | Ok
  | Er
type state = store & term_mode
```

El modo *Ok* indica que la ejecución puede continuar, mientras que *Er* indica que se ha alcanzado un estado de error. Esta distinción mecaniza los dos canales de salida *ok* y *er* introducidos en la Definición 2.4. En la presentación abstracta del Capítulo 2, la condición de salida $\epsilon \in \{ok, er\}$ aparece como un índice de la tripleta. En esta formalización se incorpora directamente en el estado. De esta manera, una aserción puede distinguir los estados normales de los erróneos inspeccionando su segundo componente.

Las expresiones del lenguaje se definen mediante un tipo inductivo:

```
type expr =
| Var : var -> expr
| Const : nat -> expr
| Plus : expr -> expr -> expr
| Minus : expr -> expr -> expr
| Times : expr -> expr -> expr
| Eq : expr -> expr -> expr
| Lt : expr -> expr -> expr
| Gt : expr -> expr -> expr
```

Las expresiones en nuestro lenguaje son puras, es decir, calcular $2+2$ no modifica la memoria ni puede fallar o arrojar múltiples resultados distintos. Por lo tanto, su semántica se modela mediante una función recursiva pura, y una función adaptadora que toma el *store* de la tupla del estado actual y la expresión y devuelve un natural:

```
let rec eval_expr' (s : store) (e : expr) : GTot nat =
  match e with
  | Var x -> s x
  | Const n -> n
  | Plus e1 e2 -> eval_expr' s e1 + eval_expr' s e2
  | Minus e1 e2 ->
    let res = eval_expr' s e1 - eval_expr' s e2 in
    if res >= 0 then res else 0
  | Times e1 e2 -> eval_expr' s e1 * eval_expr' s e2
  | Eq e1 e2 -> if eval_expr' s e1 = eval_expr' s e2
    then 1 else 0
  | Lt e1 e2 -> if eval_expr' s e1 < eval_expr' s e2
    then 1 else 0
  | Gt e1 e2 -> if eval_expr' s e1 > eval_expr' s e2
    then 1 else 0
```

```
let eval_expr (s : state) (e : expr) : GTot nat =
  eval_expr' s._1 e
```

La definición de comandos es similar, pero incluye instrucciones que modifican el estado o controlan el flujo. Al definir la sintaxis mediante un tipo inductivo, le otorgamos a F^* la garantía matemática de que es imposible que exista un comando fuera de esta lista.

```
type stmt =
| Assign : var -> expr -> stmt
| Nondet : var -> stmt
| Skip : stmt
| Error : stmt
| Assume : expr -> stmt
| Seq : stmt -> stmt -> stmt
| Choice : stmt -> stmt -> stmt
| Kleene : stmt -> stmt
```

Assign x e evalúa la expresión e y almacena su resultado en x . **Nondet** x asigna un valor arbitrario a la variable. **Skip** no modifica el estado, mientras que **Error** fuerza la terminación

errónea. **Assume** e conserva solo los caminos donde e es verdadera. **Seq** p q representa composición secuencial, **Choice** p q elección no determinista y **Kleene** p una cantidad finita y no determinista de ejecuciones de p .

La ejecución de los comandos no puede modelarse utilizando una función determinista $\text{state} \rightarrow \text{state}$. Existen dos impedimentos teóricos para construir un intérprete funcional tradicional:

1. Manejo de errores: necesitamos rastrear cuando un programa falla. Una función simple no diferencia entre un resultado exitoso y uno erróneo.
2. No determinismo: al incluir instrucciones como **Nondet** o **Choice**, un mismo estado inicial de la memoria puede derivar en varios caminos y estados finales válidos distintos.

Para resolver el primer problema, se incorporó el modo de terminación directamente dentro del state , entonces la propagación de errores ocurre de forma natural. Por otro lado, para resolver el segundo problema, abandonamos las funciones y modelamos la ejecución como una relación matemática utilizando un tipo inductivo indexado:

```
noeq
type runsto : stmt -> state -> state -> Type0
```

En la Sección 2.1, la semántica de un comando se representó mediante una relación binaria sobre estados. El tipo **runsto** constituye la mecanización de dicha relación. Un habitante del tipo **runsto** c s_0 s_1 constituye una derivación de ejecución de c desde el estado s_0 hasta s_1 . En consecuencia, constituye la contraparte mecanizada de

$$(s_0, s_1) \in \llbracket C \rrbracket_\epsilon$$

donde el canal ϵ queda determinado por el modo de terminación contenido en s_1 .

Como se explicó en la Sección 2.4, el uso del tipo inductivo indexado permite registrar en el propio tipo, el comando y los estados inicial y final de la ejecución. Cada constructor de **runsto** corresponde a una regla de la semántica operacional.

Como los *stores* son funciones, dos expresiones funcionales diferentes pueden representar el mismo *store* si coinciden en todas las variables. Para tratar estas diferencias conviene definir la equivalencia de estados:

```
unfold let state_equiv (s1 s2 : state) : prop =
  (forall x. s1._1 x == s2._1 x) /\
  s1._2 == s2._2
```

El constructor **R_Ext** de **runsto** transporta una derivación entre estados extensionalmente equivalentes. Esta regla no agrega nuevos comportamientos al lenguaje. Su función es permitir que F^* identifique como equivalentes estados cuyos *stores* coinciden, aunque hayan sido contruidos mediante expresiones funcionales sintácticamente diferentes.

```
| R_Ext : #p : stmt -> #s0 : state -> #s1 : state ->
  runsto p s0 s1 -> s0' : state -> s1' : state ->
  (#_ : squash (state_equiv s0 s0')) ->
  (#_ : squash (state_equiv s1 s1')) ->
  runsto p s0' s1'
```

La instrucción más elemental que modifica el estado es la asignación. Esta se formaliza mediante:

```
| R_Assign : s : state { s._2 == Ok } ->
  #x : var -> #e : expr ->
  runsto (Assign x e) s (override s._1 x (eval_expr s e), s._2)
```


Esta regla actúa como el puente entre el mundo puro de las expresiones y el mundo impuro de los comandos. Toma un estado de éxito arbitrario s , evalúa matemáticamente la expresión utilizando la función determinista `eval_expr s e`, y garantiza que el estado final es idéntico a s , pero con el valor de la variable x actualizado mediante `override`. Por último, como `Assign` es una operación segura en nuestro lenguaje, el modo de terminación siempre es el canal de éxito.

La asignación no determinista se representa mediante:

```
| R_Nondet : s : state { s._2 == Ok } -> #x : var -> v : value ->
  runsto (Nondet x) s (override s._1 x v, s._2)
```

Si estuviéramos programando un intérprete tradicional, implementar el no determinismo requeriría un generador de números pseudoaleatorios, lo cual ensuciaría el razonamiento matemático. En nuestra formalización, el valor v no se calcula computacionalmente, sino que se recibe como un parámetro lógico que el sistema deductivo provee. Esto le otorga al verificador la capacidad de instanciar v libremente con cualquier valor necesario para explorar una traza que conduzca a un error.

Representamos la terminación normal, de error y filtrado de caminos de la siguiente manera:

```
| R_Skip : s : state { s._2 == Ok } -> runsto Skip s s
| R_Error : s : state { s._2 == Ok } -> runsto Error s (s._1, Er)
| R_Assume : s : state { s._2 == Ok } -> #e : expr ->
  squash (eval_expr s e != 0) ->
  runsto (Assume e) s s
```

`R_Skip` conserva el estado. La regla `R_Error` conserva el *store*, pero cambia el modo de terminación. Por su parte, `R_Assume` actúa como un filtro lógico que descarta los caminos de ejecución inviables. Esta regla no produce un estado de error cuando la condición es falsa. En ese caso, no existe una derivación de `runsto`. De esta manera, los caminos que no satisfacen la condición son eliminados de la relación semántica.

La composición secuencial se divide según el modo de terminación del primer comando:

```
| R_SeqEr : #p : stmt -> #q : stmt ->
  #s : state { s._2 == Ok } -> #t : state { t._2 == Er } ->
  runsto p s t -> runsto (Seq p q) s t
| R_Seq : #p : stmt -> #q : stmt ->
  #s : state { s._2 == Ok } -> #t : state { t._2 == Ok } -> #u : state ->
  runsto p s t -> runsto q t u ->
  runsto (Seq p q) s u
```

La regla `R_Seq` establece que si la primera instrucción p termina con éxito en un estado intermedio t , la ejecución normal continúa evaluando q a partir de t hasta llegar al estado final u . Sin embargo, el comportamiento cambia en la regla `R_SeqEr`. Si la semántica garantiza que el comando p abortó dejando un estado de error t , la regla corta la ejecución. El comando secundario q nunca se menciona en las premisas, lo que significa matemáticamente que no se evalúa. El estado de error t se propaga hacia la superficie de la secuencia, garantizando la interrupción que la sub-aproximación requiere para capturar *bugs* tempranos.

La elección no determinista posee una regla para cada rama:

```
| R_ChoiceL : #p : stmt -> #q : stmt ->
  #s : state { s._2 == Ok } -> #t : state ->
  runsto p s t -> runsto (Choice p q) s t
| R_ChoiceR : #p : stmt -> #q : stmt ->
  #s : state { s._2 == Ok } -> #t : state ->
  runsto q s t -> runsto (Choice p q) s t
```

Para justificar una transición de `Choice p q` alcanza con proporcionar una derivación de una de sus ramas.

Por último, la estrella de Kleene representa una cantidad finita y no determinista de ejecuciones. El caso base realiza cero iteraciones, mientras que el caso inductivo agrega una nueva ejecución p a una repetición previa:

```
| R_Kleene0 : #p : stmt -> #s : state { s._2 == Ok } ->
  runsto (Kleene p) s s
| R_KleeneS : #p : stmt -> #s : state { s._2 == Ok } -> #t : state ->
  runsto (Seq (Kleene p) p) s t ->
  runsto (Kleene p) s t
```

3.2. La IL como tipo inductivo

La Definición 2.6 introdujo la derivabilidad como la existencia de un árbol finito construido mediante las reglas de inferencia de la IL. Utilizando la correspondencia Curry-Howard y los tipos inductivos indexados presentados en la Sección 2.4, esta noción se mecaniza mediante el tipo `il_triple`.

Una aserción puede interpretarse como un predicado sobre el conjunto de estados. Dado que en la Sección 3.1 dicho conjunto fue instanciado mediante el tipo `state`, las aserciones se representan en F^* como:

```
type cond = state -> prop
```

A partir de esta definición, la derivabilidad de una tripleta se representa mediante:

```
noeq type il_triple : (pre : cond) -> (p : stmt) -> (post : cond) -> Type
```

Un valor de tipo `il_triple pre p post` es un árbol de derivación construido mediante las reglas del sistema. Por lo tanto, `il_triple` mecaniza la noción sintáctica de derivabilidad de la Definición 2.6, no la validez semántica de la Definición 2.4. La relación entre ambas nociones será establecida en la Sección 3.4 mediante la función `soundness`.

Cada constructor de `il_triple` corresponde a una de las reglas de inferencia presentadas en la Figura 2.1. Del mismo modo que un término de `runsto p s0 s1` constituye un testigo de ejecución, un término de `il_triple pre p post` constituye un testigo de derivabilidad.

Las precondiciones de las reglas describen estados desde los cuales la ejecución todavía puede continuar, es decir, es correcta. Para expresarlo se utiliza:

```
unfold let is_ok (c : cond) : cond = fun s -> c s /\ s._2 == Ok
```

Así, `is_ok pre` exige que el estado satisfaga `pre` y también que su modo de terminación sea `Ok`. A continuación se analiza cómo se codifican estructuralmente las reglas de la teoría.

En primer lugar, observamos las instrucciones más simples de nuestro lenguaje:

```
| I_Skip : pre : cond ->
  il_triple (is_ok pre) Skip (is_ok pre)
| I_Error : pre : cond ->
  il_triple (is_ok pre) Error
  (fun s -> let (st, m) = s in m == Er /\ pre (st, Ok))
| I_Assume : pre : cond -> #e : expr ->
  il_triple (is_ok pre) (Assume e)
  (is_ok (fun s -> pre s /\ eval_expr s e != 0))
```

La regla `I_Skip` conserva la aserción. Por el contrario, la regla `I_Error` modela la interrupción del flujo normal. Conserva el *store* descrito por la presunción, pero cambia el modo de terminación. `I_Assume` incorpora la condición evaluada al resultado. La postcondición conserva solo los estados que satisfacen la presunción y para los cuales la expresión es verdadera.

Como se mencionó en la introducción, la sub-aproximación requiere descartar el axioma de sustitución hacia atrás de la HL, ya que este es inválido para garantizar la alcanzabilidad. En su

lugar, la IL exige el uso del razonamiento hacia adelante. La regla de asignación debe conservar suficiente información histórica para justificar la alcanzabilidad del estado final:

```
| I_Assign : #pre : cond -> #x : var -> #e : expr ->
  il_triple
    (is_ok pre) (Assign x e)
    (is_ok (fun s -> exists x_init.
      pre ((override s._1 x x_init), s._2) /\
      (s._1 x = eval_expr ((override s._1 x x_init), s._2) e)))
```

El valor existencial x_init representa el valor que tenía x antes de la asignación. A partir del *store* final se reconstruye un posible *store* inicial reemplazando el valor actual de x por x_init . La primera conjunción exige que este estado reconstruido satisfaga **pre**. La segunda establece que el valor actual de x coincide con la evaluación de e en dicho estado inicial.

La regla **I_Nondet** usa una construcción similar. En este caso, para cada estado final se exige la existencia de algún valor anterior de x que permita reconstruir un estado inicial que satisfaga la presunción.

```
| I_Nondet : #x : var -> #pre : cond ->
  il_triple (is_ok pre) (Nondet x)
    (is_ok (fun s -> exists v. pre ((override s._1 x v), s._2)))
```

La composición secuencial demuestra el poder composicional de manejar ambos flujos en un solo canal de estados:

```
| I_Seq : #p : stmt -> #q : stmt ->
  #pre : cond -> #mid : cond -> #post : cond ->
  il_triple (is_ok pre) p mid ->
  il_triple (is_ok mid) q post ->
  il_triple (is_ok pre) (Seq p q)
    (fun s -> post s /\ (s._2 == Er /\ mid s))
```

La postcondición reúne dos clases de resultados. **post s** contiene los estados obtenidos después de ejecutar tanto p como q . La segunda disyunción contiene los estados de error producidos durante p , en los cuales q no se ejecuta. Cuando p finaliza normalmente, el estado intermedio satisface **mid** y puede utilizarse como presunción de q . Cuando p finaliza en **Er**, el error se propaga directamente al resultado de la secuencia.

La IL permite justificar una ejecución no determinista seleccionando solo una rama:

```
| I_ChoiceL : #p : stmt -> #q : stmt ->
  #pre : cond -> #post : cond ->
  il_triple (is_ok pre) p post ->
  il_triple (is_ok pre) (Choice p q) post
| I_ChoiceR : #p : stmt -> #q : stmt ->
  #pre : cond -> #post : cond ->
  il_triple (is_ok pre) q post ->
  il_triple (is_ok pre) (Choice p q) post
```

A diferencia de un análisis que intenta cubrir todos los caminos, una derivación que se da por una sub-aproximación puede centrarse en una única rama capaz de alcanzar el resultado deseado. Esta posibilidad no significa que la lógica determine automáticamente qué rama conduce a un error. En la formalización, la rama relevante es seleccionada explícitamente al construir el árbol de derivación.

El caso de las iteraciones finitas se representa mediante:

```
| I_Kleene0 :
  #p : stmt -> #pre : cond ->
  il_triple (is_ok pre) (Kleene p) (is_ok pre)
| I_KleeneS : #p : stmt -> #pre : cond -> #post : cond ->
```

```

il_triple (is_ok pre) (Seq (Kleene p) p) post ->
il_triple (is_ok pre) (Kleene p) post

```

Además, la formalización incluye una regla paramétrica basada en una familia de aserciones:

```

| I_KleeneVariant :
#variant : (nat -> cond) -> #p : stmt ->
(n : nat -> il_triple (is_ok (variant n)) p (variant (n + 1))) ->
il_triple (kleene_pre variant) (Kleene p) (kleene_post variant)

```

La premisa demuestra que cada estado perteneciente a `variant (n + 1)` puede reconstruirse a partir de un estado perteneciente a `variant n`. Repitiendo este argumento es posible obtener una traza finita que comienza en `variant 0`.

Como se observó en el Lema 2.2, la post-imagen preserva uniones. Esta propiedad se refleja mediante:

```

| I_Disjunction : #pre1 : cond -> #pre2 : cond ->
#p : stmt -> #post1 : cond -> #post2 : cond ->
il_triple (is_ok pre1) p post1 ->
il_triple (is_ok pre2) p post2 ->
il_triple (is_ok (fun s -> pre1 s \/ pre2 s)) p
(fun s -> post1 s \/ post2 s)

```

La regla permite combinar derivaciones obtenidas a partir de conjuntos diferentes de estados iniciales.

3.3. La Regla de Consecuencia

Una de las diferencias estructurales entre la IL y la verificación clásica es el comportamiento de su Regla de Consecuencia. Como vimos en la Sección 2.1, la sub-aproximación exige invertir la dirección de las implicaciones. El sistema permite debilitar la precondition y fortalecer la postcondition. Esto es para no exigir estados iniciales innecesarios y para enfocar el análisis en un subconjunto de estados finales de interés, descartando lógicamente otras ramas. La validez de estas direcciones se justificó utilizando la monotonía de la post-imagen establecida en el Lema 2.2.

Si una tripleta satisface $\text{post} \subseteq \text{post}'(C)(\text{pre})$, entonces las condiciones $\text{pre} \subseteq \text{pre}'$ y $\text{post}' \subseteq \text{post}$ permiten concluir:

$$\text{post}' \subseteq \text{post}'(C)(\text{pre}')$$

Esta regla se codifica mediante:

```

| I_Consequence : #pre : cond -> #p : stmt ->
#post : cond -> pre' : cond -> post' : cond ->
il_triple (is_ok pre) p post ->
squash (forall x. pre x ==> pre' x) ->
squash (forall x. post' x ==> post x) ->
il_triple (is_ok pre') p post'

```

La clave en esta formalización está en haber envuelto estas proposiciones universales dentro del `squash`. Esto hace que la formalización solo necesite conocer la validez de las condiciones laterales, sin necesidad de inspeccionar la estructura computacional de sus demostraciones. `F*` transforma estas condiciones en obligaciones de verificación que, en muchos casos, pueden ser resueltas automáticamente por `Z3`.

Sin embargo, esta automatización es local. El *solver* puede comprobar implicaciones proposicionales, igualdades y restricciones aritméticas, pero no construye por sí mismo la derivación principal de la IL. El usuario debe elegir las reglas, proporcionar aserciones intermedias y definir el camino de ejecución que se quiere justificar.

3.4. El Teorema de Soundness

El último paso para completar la formalización de la IL es demostrar que el sistema deductivo es sólido respecto a la semántica. En el marco teórico, esto se vio en el Teorema 2.7. Sin embargo, enunciamos otra vez el teorema en base a nuestra formalización.

Teorema 3.1 (Soundness de la IL formalizada). *Para todo comando p y aserciones pre y $post$, si existe una derivación*

$$\vdash_{IL} [pre] \ p \ [post]$$

entonces, para todo estado final s_1 que satisface $post$, existe un estado inicial s_0 que satisface pre y una ejecución operacional de p desde s_0 hasta s_1 :

$$\forall s_1. post \ s_1 \implies \exists s_0. pre \ s_0 \wedge runsto \ p \ s_0 \ s_1$$

En F^* codificamos este teorema de forma constructiva mediante la siguiente función con tipos dependientes:

```
let rec soundness
  (p : stmt) (pre : cond) (post : cond)
  (pf : il_triple pre p post) (s1 : state { post s1 })
  : GTot (s0 : state { pre s0 } & runsto p s0 s1) (decreases pf)
```

La firma constituye una traducción directa del Lema 2.5:

- $s_1 : state \{ post \ s_1 \}$ representa un estado final $\sigma_q \in q$;
- $s_0 : state \{ pre \ s_0 \}$ representa un estado inicial testigo $\sigma_p \in p$;
- $runsto \ p \ s_0 \ s_1$ representa la transición $(\sigma_p, \sigma_q) \in \llbracket C \rrbracket_\epsilon$.

La función conecta así los dos niveles mecanizados en las secciones anteriores. Consume un habitante de `il_triple`, que certifica derivabilidad, y produce un habitante de `runsto`, que certifica la existencia de una ejecución.

La demostración se realiza por inducción estructural sobre `pf`. La cláusula `decreases pf` indica que cada llamada recursiva se realiza sobre una subderivación menor.

Hay instrucciones en las que la traza se reconstruye de forma directa. En el caso de `Skip`, el estado inicial coincide con el final:

```
| I_Skip _ ->
  let s0 = s1 in
  let r = R_Skip s0 in
  (|s0, r|)
```

Para `Error`, el *store* inicial coincide con el final, pero el modo de terminación difiere:

```
| I_Error _ ->
  let (st1, _) = s1 in
  let s0 = (st1, Ok) in
  let r = R_Error s0 in
  (|s0, r|)
```

En `Assume`, la postcondición ya contiene la prueba de que la expresión es verdadera:

```
| I_Assume pre #e ->
  let s0 = s1 in
  let r = R_Assume s0 #e () in
  (|s0, r|)
```

Las reglas de asignación y no determinismo contienen cuantificadores existenciales en sus postcondiciones. Para construir la ejecución operacional es necesario extraer sus testigos.

En el caso de asignación, la postcondición garantiza la existencia de un valor histórico `x_init`. Sin embargo, a nivel computacional en F^* , no es posible ejecutar hacia atrás para adivinar cuál era ese valor en el *store* original. Para resolver esto nos apoyamos en la operación fantasma `indefinite_description_ghost`. Esta operación fantasma de elección clásica nos permite transformar la existencia lógica de un estado en un testigo que podemos manipular en el código de la demostración. Este testigo no forma parte de un programa ejecutable, solo se usa durante la demostración:

```
| I_Assign #pre #x #e ->
  let (st1, m) = s1 in
  let x_init = FStar.IndefiniteDescription.indefinite_description_ghost
    _ (fun x_init -> pre ((override st1 x x_init), m)
    /\ (st1 x = eval_expr ((override st1 x x_init), m) e))
  in
  let st0 = override st1 x x_init in
  let s0 = (st0, 0k) in
  let pf0 = R_Assign s0 #x #e in
  let pf1 = R_Ext pf0 s0 s1 in
  (|s0, pf1|)
```

Una vez que tenemos el *store* histórico, construimos la traza `R_Assign`. Para satisfacer la comprobación de tipos dependientes al atar el paso operacional con el estado final `s1`, invocamos a la regla de extensión semántica `R_Ext`, la cual nos permite ajustar sutiles equivalencias de extensionalidad de funciones sobre el *store*.

El caso de `Nondet` sigue el mismo patrón. La postcondición proporciona un valor histórico que permite reconstruir un estado inicial, mientras que la ejecución se instancia con el valor final de `x`.

La demostración para la secuenciación de comandos `I_Seq` ilustra cómo la sub-aproximación captura flujos anormales. La regla deductiva afirma que el resultado de `Seq p q` puede derivar del éxito final de `q` o de un fallo prematuro en `p`. Por lo tanto, el teorema de soundness debe inspeccionar el estado final `s1` para decidir qué fragmento de la semántica reconstruir:

```
| I_Seq #p #q #pre #mid #post pf_p pf_q ->
  if t2b (post s1) then
    let (|s_mid, r_q|) = soundness q (is_ok mid) post pf_q s1 in
    let (|s0, r_p|) = soundness p (is_ok pre) mid pf_p s_mid in
    let r = R_Seq #p #q #s0 #s_mid #s1 r_p r_q in
    (|s0, r|)
  else (
    let (|s0, r_p|) = soundness p (is_ok pre) mid pf_p s1 in
    let r = R_SeqEr #p #q #s0 #s1 r_p in
    (|s0, r|)
  )
```

Si `s1` satisface `post`, el estado fue obtenido después de ejecutar `q`. La reconstrucción ocurre en orden inverso a la ejecución, primero se aplica la hipótesis inductiva a `q`, obteniendo el estado intermedio `s_mid`, luego se aplica a `p`, obteniendo el estado inicial `s0`. Finalmente, ambas derivaciones se componen mediante `R_Seq`.

Si `s1` no satisface `post`, la postcondición de `I_Seq` garantiza que se trata de un estado de error producido por `p`. En ese caso, se reconstruye únicamente la ejecución del primer comando y se utiliza `R_SeqEr` para propagar el fallo.

Los casos de elección aplican la hipótesis inductiva solo en la rama seleccionada por la derivación:

```

| I_ChoiceL #p #q #pre #post pf_p ->
  let (ls0, r_pl) = soundness p (is_ok pre) post pf_p s1 in
  let r = R_ChoiceL #p #q #s0 #s1 r_p in
  (ls0, r_l)

```

El caso derecho es análogo. No es necesario reconstruir una ejecución de la rama descartada.

Para `I_Kleene0`, la ejecución realiza cero iteraciones. Para `I_KleeneS`, la hipótesis inductiva reconstruye una ejecución de `Seq (Kleene p) p`, que luego se transforma en una ejecución de `Kleene p` mediante `R_KleeneS`.

El tratamiento de la iteración mediante `I_KleeneVariant` usa un mecanismo híbrido. Primero, utiliza la descripción indefinida para encontrar el número de iteraciones n en el que finalizó el bucle. Luego, construye la traza completa declarando una función auxiliar recursiva sobre el número de iteraciones. El caso base utiliza `R_Kleene0`. En el caso inductivo, se reconstruye primero la última ejecución de `p`, luego a repetición anterior y finalmente se componen ambas mediante `R_Seq` y `R_KleeneS`.

En `I_Consequence`, el estado final que satisface `post'` también satisface `post` por la condición lateral de fortalecimiento. Luego se aplica la hipótesis inductiva a la derivación original. El estado inicial obtenido satisface `pre` y, por debilitamiento, también satisface `pre'`.

Para `I_Disjunction`, la prueba inspecciona cuál de las dos postcondiciones satisface el estado final y aplica la hipótesis inductiva a la derivación correspondiente.

La aceptación de la función `soundness` por F^* instancia el Teorema 2.7 para el lenguaje concreto de este capítulo. En particular, construye los testigos exigidos por la caracterización del Lema 2.5:

$$\text{il_triple } pre \ p \ post \implies \forall s_1 \in post. \exists s_0 \in pre. \text{runsto } p \ s_0 \ s_1$$

Por lo tanto, el Corolario 2.8 también se aplica a esta formalización, ya que si una derivación produce un resultado de error satisfacible, existe un estado inicial y una ejecución operacional que alcanza dicho error.

Esta garantía debe entenderse respecto del lenguaje y de la semántica formalizados. F^* certifica que el sistema deductivo no introduce estados finales sin respaldo operacional dentro del modelo definido. Sobre esta base, el próximo capítulo extenderá el estado con memoria dinámica y trasladará la misma estrategia de mecanización a la ISL.

Capítulo 4

Formalización de ISL

“The second, to divide each of the difficulties I examined into as many parts as possible and as may be required in order to resolve them better.”

— René Descartes

En el capítulo anterior se mecanizó la IL sobre un lenguaje imperativo sin memoria dinámica. Sin embargo, el modelo de memoria que se utilizó es insuficiente para analizar programas realistas. Al introducir referencias de memoria dinámica surge el problema de *aliasing*.

En este capítulo presentamos la formalización mecanizada de la ISL. No se reexplicarán los fundamentos de la sub-aproximación, los cuales se mantienen idénticos al capítulo anterior, sino detallar cómo la arquitectura de nuestra formalización en F^* debió evolucionar para incorporar el razonamiento local sobre porciones disjuntas de memoria, la conjunción separadora y la detección de *bugs* especiales como el *Use After Free*.

4.1. Extensión del lenguaje y del modelo de estado

Para soportar el razonamiento local propuesto por Reynolds [19] en la SL, y su posterior extensión en la ISL, debemos abandonar la premisa de que un programa conoce y opera sobre la totalidad de la memoria. En el Capítulo 3, el estado estaba formado por un *store* y un modo de terminación. Ese modelo era suficiente para razonar sobre asignaciones, no determinismo y control de flujo, pero no permitía representar memoria dinámica ni *aliasing*.

Para aislar y resolver las colisiones de memoria, el estado del programa se divide lógicamente en dos componentes independientes. Como se vio en la Sección 2.2, el primero de estos es el *store*, que mapea variables a valores, y el segundo es el *heap*, que mapea direcciones físicas a los valores allí almacenados. En consecuencia, el estado utilizado para la formalización de la ISL se redefine como una terna formada por un *store*, un *heap* y un modo de terminación.

En primer lugar, se incorporan las ubicaciones de memoria y se amplía el tipo de los valores:

```
type var = string
type loc = nat
type value =
  | Nat of nat
  | Loc of loc
type store = var -> value
```

A diferencia del Capítulo 3, donde todos los valores eran números naturales, ahora se distingue entre valores numéricos y direcciones de memoria. El constructor `Nat n` representa el valor natural `n`, mientras que `Loc l` representa la ubicación `l`. La dirección 0 se reserva para modelar el puntero nulo, por lo que las reglas correspondientes a accesos exitosos exigen direcciones no nulas.

La Definición 2.16 presentó los *heaps* de la ISL como funciones parciales cuyo codominio incluye tanto valores como una marca negativa $\perp$. Para modelar esta parcialidad en F* se decidió codificar como una función total hacia un nuevo tipo de dato `cell`, el cual explicita las tres situaciones posibles de una celda:

```
type cell =
  | Full of value
  | Empty
  | Unknown
type heap = loc -> cell
```

Cada uno de los constructores de `cell` define cuál es el estado actual de la celda de memoria:

1. **Full** v : indica que la ubicación es conocida, está asignada y contiene el valor v .
2. **Empty**: indica que la ubicación es conocida, pero fue liberada y todavía no fue reasignada.
3. **Unknown**: indica que el fragmento local del *heap* no tiene información sobre esa dirección.

El constructor **Empty** constituye la representación mecanizada del elemento negativo $\perp$ introducido en la Definición 2.16. Por otro lado, **Unknown** permite representar la parcialidad del *heap* dentro de una función total.

Es importante distinguir **Empty** de **Unknown**. Una ubicación marcada como **Empty** pertenece al fragmento conocido de memoria y registra que fue liberada. En cambio, una ubicación marcada como **Unknown** se encuentra fuera del fragmento local descrito y puede formar parte de otro *heap* disjunto agregado mediante un marco.

En este caso, la liberación de memoria transforma una celda **Full** v en **Empty**, pero no la convierte en **Unknown**. De esta manera, la formalización conserva la información de que la ubicación es conocida incluso después de haber sido liberada. Esta decisión de representación se corresponde con la intuición de monotonía presentada en el Lema 2.18, donde una ubicación conocida no deja de ser conocida como consecuencia de una liberación.

El estado completo queda definido como:

```
type state = store & heap & term_mode
type cond = state -> prop
```

4.2. Álgebra de *heaps* y aserciones espaciales

Para representar la conjunción separadora de la Definición 2.13 es necesario definir cuándo dos *heaps* representan fragmentos independientes y cómo puede construirse su unión. Para el primer caso, como los *heaps* se representan como funciones totales, la disjunción se define punto a punto. Dos celdas son disjuntas cuando al menos una de ellas no contiene información local:

```
let cell_disjoint (c1 c2 : cell) : prop =
  c1 == Unknown \ / c2 == Unknown
```

Luego, dos *heaps* son disjuntos cuando sus celdas son disjuntas para toda ubicación:

```
let heaps_disjoint (h1 h2 : heap) : prop =
  forall l. cell_disjoint (h1 l) (h2 l)
```

Esta definición mecaniza la relación $h_1 \perp h_2$ de la Definición 2.10. En el modelo matemático del Capítulo 2, la disjunción se expresaba mediante la separación de los dominios de dos funciones parciales. En la representación de F*, se exige que no exista una ubicación conocida simultáneamente por ambos *heaps*.

Si dos celdas son disjuntas, su unión selecciona la celda que contiene información:

```

let cell_union (c1 c2 : cell { cell_disjoint c1 c2 }) : cell =
  match c1 with
  | Unknown -> c2
  | _ -> c1

```

En esta formalización, el constructor `Unknown` actúa como elemento neutro. El caso restante puede devolver `c1` porque el refinamiento `cell_disjoint c1 c2` garantiza que, cuando `c1` es conocida, `c2` es necesariamente `Unknown`. Luego, la unión de *heaps* se define punto a punto:

```

let heap_union (h1 h2 : heap { heaps_disjoint h1 h2 }) : heap =
  fun l -> cell_union (h1 l) (h2 l)

```

El refinamiento exige una prueba de disjunción antes de construir la unión. Por lo tanto, `heap_union h1 h2` representa la unión disjunta $h_1 \uplus h_2$ definida en la Sección 2.2.

Para construir las especificaciones de los programas, formalizamos los tres predicados espaciales fundamentales que definimos en 2.11, 2.12 y 2.17. El más simple es `emp`, el cual afirma que la huella de memoria actual está completamente vacía. Matemáticamente esto significa que el programa no tiene permisos sobre ninguna celda, por lo que toda locación debe ser evaluada como `Unknown`:

```

let emp : cond =
  fun (st, hp, m) -> forall l. hp l == Unknown

```

Por otro lado, cuando un programa aloja memoria o accede a un puntero, necesitamos describir celdas específicas. Para esto definimos los predicados `points_to` y `points_to_empty`:

```

let points_to (l : loc) (v : value) : cond =
  fun (st, hp, m) -> l != 0 /\ hp l == Full v /\
    forall l'. (l' <> l) ==> (hp l' == Unknown)

let points_to_empty (l : loc) : cond =
  fun (st, hp, m) -> l != 0 /\ hp l == Empty /\
    forall l'. (l' <> l) ==> (hp l' == Unknown)

```

En el primero, se mecaniza la aserción positiva que describe exactamente una celda activa. La condición fuerza que todas las ubicaciones diferentes de `l` sean `Unknown`. Por lo tanto, `points_to l v` no describe un *heap* arbitrariamente grande que contiene esa celda, sino un fragmento local formado exactamente por ella. Esto corresponde a la interpretación exacta del `points-to` de la Definición 2.12. En el segundo caso, se está representando a la aserción negativa. `points_to_empty l` mecaniza la aserción de *heap* negativo de la Definición 2.17. Expresa que `l` es una ubicación conocida, no nula, que fue liberada y todavía no fue reasignada.

A partir del álgebra anterior, la conjunción separadora se representa mediante:

```

unfold
let sep_conj (p q : cond) : cond =
  fun (st, hp, m) -> exists h1 h2.
    heaps_disjoint h1 h2 /\
    hp == heap_union h1 h2 /\
    p (st, h1, m) /\ q (st, h2, m)

unfold let ( ** ) = sep_conj

```

Esta definición constituye la traducción directa de la Definición 2.13. Para que un estado satisfaga `p ** q`, deben construirse dos *heaps* disjuntos cuya unión sea el *heap* original, de forma que `p` se satisfaga sobre el primer fragmento y `q` sobre el segundo. Al obligar a las aserciones a razonar sobre particiones estrictamente independientes de la memoria, el operador `**` prepara el terreno para la introducción de la Regla del Marco y de las huellas locales que se introducirán más adelante.

4.3. Semántica operacional y metateoría

La relación `runsto` conserva la estructura introducida en la Sección 3.1. La diferencia en este caso es que opera sobre estados formados por un *store*, un *heap* y un modo de terminación. Los constructores correspondientes a asignación, no determinismo, secuencia, elección y estrella de Kleene mantienen el mismo comportamiento general. La extensión principal consiste en incorporar las operaciones `Alloc`, `Free`, `Load` y `Store`, introducidas teóricamente en la Sección 2.2.

```
type stmt =
| Assign : var -> expr -> stmt
| Nondet : var -> stmt
| Skip : stmt
| Error : stmt
| Assume : expr -> stmt
| Seq : stmt -> stmt -> stmt
| Choice : stmt -> stmt -> stmt
| Kleene : stmt -> stmt
| Alloc : var -> stmt
| Free : expr -> stmt
| Load : var -> expr -> stmt
| Store : expr -> expr -> stmt
```

Como tanto el *store* como el *heap* se representan mediante funciones, la equivalencia de estados del Capítulo 3 debe apilarse para comparar ambos componentes punto a punto:

```
unfold let state_equiv (s1 s2 : state) : prop =
  (forall x. s1._1 x == s2._1 x) /\
  (forall l. s1._2 l == s2._2 l) /\
  s1._3 == s2._3
```

Analizamos los constructores operacionales que difieren de aquellos vistos en el Capítulo 3. La regla operacional de reserva es:

```
| R_Alloc : s : state { s._3 == Ok } -> #x : var ->
  l : loc { l != 0 } -> v : value ->
  #(squash (s._2 l == Unknown \/ s._2 l == Empty)) ->
  runsto (Alloc x) s (override s._1 x (Loc l), override s._2 l (Full v), s._3)
```

La ubicación seleccionada debe ser no nula. Además, debe ser una dirección sobre la cual el *heap* no poseía información o una dirección conocida que se encontraba liberada. El primer caso representa la reserva de una ubicación nueva respecto del fragmento local. El segundo representa la reasignación de una celda negativa. Esta distinción corresponde a las reglas `ALLOC1` y `ALLOC2` de la Figura 2.5.

La liberación exitosa se formaliza mediante:

```
| R_Free : s : state { s._3 == Ok } -> e : expr -> l : loc -> v : value ->
  #(squash (eval_expr s e == 1 /\ l != 0)) ->
  #(squash (s._2 l == Full v)) ->
  runsto (Free e) s (s._1, override s._2 l Empty, s._3)
```

La expresión debe evaluar una dirección no nula que contenga una celda activa. La ejecución no elimina la ubicación del *heap*, sino que transforma `Full v` en `Empty`. De esta manera, se preserva el recurso negativo requerido por la ISL.

Si la dirección ya se encontraba liberada, se produce un *double free*. En este caso, el *heap* se conserva, ya que la ubicación continúa estando liberada, pero el modo cambia a `Er`.

```
| R_FreeEr : s : state { s._3 == Ok } -> e : expr -> l : loc { l != 0 } ->
  #(squash (eval_expr s e == 1)) ->
```

```
#(squash (s._2 l == Empty)) ->
runsto (Free e) s (s._1, s._2, Er)
```

El intento de liberar la dirección nula se trata mediante un constructor separado:

```
| R_FreeNull : s : state { s._3 == 0k } -> e : expr ->
#(squash (eval_expr s e == 0)) ->
runsto (Free e) s (s._1, s._2, Er)
```

Las reglas de lectura siguen el mismo patrón. Una lectura exitosa requiere una celda activa y actualiza el *store* con su contenido:

```
| R_Load : s : state { s._3 == 0k } -> x : var -> e : expr ->
l : loc { l != 0 } -> v : value ->
#(squash (s._2 l == Full v)) ->
#(squash (eval_expr s e == l)) ->
runsto (Load x e) s (override s._1 x v, s._2, s._3)
```

Si la dirección contiene *Empty*, la ejecución representa un acceso posterior a una liberación y finaliza en modo *Er*. El acceso a la dirección nula se modela de forma análoga mediante *R_LoadNull*.

La escritura exitosa requiere que la ubicación se encuentre activa y reemplaza su contenido:

```
| R_Store : s : state { s._3 == 0k } -> e1 : expr -> e2 : expr ->
l : loc { l != 0 } -> v : value ->
#(squash (s._2 l == Full v)) ->
#(squash (eval_expr s e1 == l)) ->
runsto (Store e1 e2) s (s._1, override s._2 l (Full (Nat (eval_expr s e2))), s._3)
```

Los constructores *R_StoreEr* y *R_StoreNull* representan, respectivamente, una escritura sobre una celda liberada y una escritura sobre la dirección nula.

La separación entre los casos exitosos, negativos y nulos refleja directamente las reglas de memoria de la Figura 2.5. El tipo *term_mode* solo distingue entre *0k* y *Er*. La causa específica del error queda registrada en el constructor utilizado para construir la derivación operacional.

La propiedad central del razonamiento local es que una ejecución sobre un fragmento de memoria puede extenderse con un *heap* disjunto que permanece inalterado. Esta propiedad fue presentada de forma abstracta en el Lema 2.15. En la formalización se mecaniza con la función *r_frame*:

```
let rec r_frame (#p : stmt) (#s0 : state) (#s1 : state)
(r : runsto p s0 s1) (h_fr : heap)
(#_ : squash (let (_, hp, _) = s0 in heaps_disjoint hp h_fr))
(#_ : squash (let (_, hp, _) = s1 in heaps_disjoint hp h_fr)) :
GTot (runsto p
  (let (st, hp, m) = s0 in (st, heap_union hp h_fr, m))
  (let (st, hp, m) = s1 in (st, heap_union hp h_fr, m)))
(decreases r)
```

La función recibe la derivación operacional local y un *heap* adicional *h_fr*. Las dos premisas de disjunción garantizan que el marco no se superponga con los *heaps* inicial y final. Como resultado se obtiene una nueva derivación:

$$\text{runsto } p (s_0 \bullet h_{fr}) (s_1 \bullet h_{fr}),$$

donde el *store* y el modo se conservan, mientras que el mismo marco se agrega a ambos *heaps*. La demostración se hace por inducción estructural sobre *r*. Para cada constructor *runsto* se reconstruye la misma ejecución sobre los *heaps* extendidos.

El caso secuencial requiere conocer que el marco también es disjunto del *heap* intermedio. Para obtener esta propiedad se utiliza el lema:

```

let rec lemma_runsto_disjoint (#p : stmt) (#s0 : state) (#s1 : state)
  (h_fr : heap) (r : runsto p s0 s1) :
  Lemma (requires (let (_, hp1, _) = s1 in heaps_disjoint hp1 h_fr))
    (ensures (let (_, hp0, _) = s0 in heaps_disjoint hp0 h_fr))
    (decreases r)

```

Este lema transporta hacia atrás la disjunción con el marco. En el caso de una secuencia, primero se aplica la ejecución del segundo comando para obtener la disjunción con el *heap* intermedio y luego a la ejecución del primero.

La función `r_frame` demuestra la dirección local a global de la propiedad de localidad. Luego, será reutilizada tanto en la prueba de *soundness* de la Regla del Marco como en la demostración del Teorema de la Huella 2.23.

4.4. La ISL como tipo inductivo

Estamos en condiciones de mecanizar el sistema deductivo de la ISL. Al igual que en el Capítulo 3, representamos la relación de derivabilidad mediante un tipo inductivo en F^* , al que denominaremos `isl_triple`. Su firma mantiene la estructura fundamental de la sub-aproximación, separando los canales de éxito y error, pero ahora sus precondiciones y postcondiciones evalúan la forma del *heap*:

```

noeq type isl_triple : (pre : cond) -> (p : stmt) -> (post : cond) -> Type

```

Los constructores correspondientes a la asignación, no determinismo, secuencia, elección, consecuencia, disyunción y estrella de Kleene mantienen la estructura presentada para la IL. Las diferencias se centran en las reglas de memoria dinámica y en la incorporación de la Regla del Marco.

El poder de los *heaps* negativos se ilustra al analizar la instrucción de liberación de memoria correspondiente a `Free`. En nuestra formalización, el comportamiento de esta instrucción se divide en constructores independientes según su canal de terminación.

La regla para la liberación exitosa refleja la mutación de una celda activa hacia un *heap* negativo. Tras la ejecución de `Free e`, la postcondición asegura que la locación evaluada se encuentra vacía. Sin embargo, para garantizar que este estado proviene de uno válido, la aserción utiliza el cuantificador existencial `exists v` para reconstruir el estado histórico. Afirma que, si tomáramos el estado actual y restauráramos la celda liberada con el valor original, obtendríamos exactamente el estado histórico que satisfacía la precondición original:

```

| ISL_Free : #pre : cond -> e : expr ->
  isl_triple (is_ok pre) (Free e)
    (is_ok (fun s -> exists (v : value).
      points_to_empty (eval_expr s e) s /\
      pre (s._1, override s._2 (eval_expr s e) (Full v), s._3)))

```

Al preservar el *heap* negativo como un recurso, la ISL habilita la detección deductiva de errores espaciales. Si el programa intentase liberar un puntero que ya ha sido liberado previamente, la semántica intercepta la violación de tipo *double free*:

```

| ISL_FreeEr : #pre : cond -> e : expr ->
  isl_triple (is_ok pre) (Free e)
    (fun s -> s._3 == Er /\
      points_to_empty (eval_expr s e) s /\
      pre (s._1, s._2, Ok))

```

Finalmente, modelamos la desreferencia de punteros nulos:

```

| ISL_FreeNode : #pre : cond -> e : expr ->
  isl_triple (is_ok pre) (Free e)
    (fun s -> s._3 == Er /\ eval_expr s e == 0 /\ pre (s._1, s._2, Ok))

```

Las familias de **Load** y **Store** siguen la misma estructura. Las reglas exitosas reconstruyen el *store* o el *heap* anterior mediante valores existenciales, mientras que las reglas de error conservan la celda negativa o la condición de nulidad que explica la transición a **Er**.

Por otro lado, la Regla del Marco debe preservar no solo el *heap* adicional, sino también la validez de la aserción enmarcada respecto del *store*. Para expresar esta condición se calcula el conjunto de variables que un comando puede modificar:

```
let rec modifies (p : stmt) (x : var) : prop =
  match p with
  | Assign y _ -> x = y
  | Nondet y -> x = y
  | Seq s1 s2 -> modifies s1 x /\ modifies s2 x
  | Choice s1 s2 -> modifies s1 x /\ modifies s2 x
  | Kleene s -> modifies s x
  | Alloc y -> x = y
  | Load y _ -> x = y
  | _ -> False
```

Luego, definimos cuándo una aserción es semánticamente inmune a la mutación de ciertas variables. Dos *stores* coinciden fuera de un conjunto de variables cuando asignan los mismos valores a todas las variables no incluidas en dicho conjunto. Basándonos en esto, una aserción es independiente de un conjunto de variables si su evaluación lógica devuelve el mismo resultado sin importar cómo cambien los valores de dichas variables:

```
let match_except_vars (vars : string -> prop) (st1 st2 : store) : prop =
  forall x. ~(vars x) ==> st1 x == st2 x

let independent_on_vars (vars : string -> prop) (c : cond) : prop =
  forall (st1 st2 : store) (hp : heap) (m1 m2 : term_mode).
    match_except_vars vars st1 st2 ==> (c (st1, hp, m1) <==> c (st2, hp, m2))
```

La definición exige que la verdad de *c* no cambie cuando se modifican las variables incluidas en *vars*. Además, exige que la aserción del marco no dependa del modo de terminación, ya que compara su validez para modos arbitrarios *m1* y *m2*. Esta condición permite preservar el marco incluso cuando una ejecución pasa del canal normal al canal de error.

Con esto, la Regla del Marco se codifica como el siguiente constructor:

```
| ISL_Frame : #pre : cond -> #p : stmt -> #post : cond -> fr : cond ->
  isl_triple (is_ok pre) p post ->
  squash (independent_on_vars (modifies p) fr) ->
  isl_triple (is_ok (pre ** fr)) p (post ** fr)
```

La conjunción separadora protege el *heap* del marco, ya que exige que el fragmento descrito por *fr* sea disjunto de la memoria local utilizada por la derivación. La condición que provee *independent_on_vars* protege el *store*, garantizando que las modificaciones permitidas por *p* no alteran la validez de *fr*.

La regla *ISL_Frame* actúa como un elevador de contexto y representa la regla deductiva de la ecuación 2.1. Toma como premisa implícita una tripleta previamente demostrada que opera sobre un fragmento local de memoria descrito por *pre* y *post*. Luego, recibe como parámetro lógico un predicado arbitrario *fr*, el cual describe el conjunto de recursos adicionales.

4.5. El Teorema de Soundness

El Teorema de Soundness 2.19 establece que toda tripleta derivable de la ISL es semánticamente válida. Su mecanización conserva la misma forma general que la función de *soundness* desarrollada para IL en el Capítulo 3.

Teorema 4.1 (Soundness de la ISL formalizada). *Para todo comando p , presunción pre y resultado $post$, si existe una derivación*

$$\vdash_{\text{ISL}} [pre] \ p \ [post]$$

entonces, para todo estado final s_1 que satisface $post$, existe un estado inicial s_0 que satisface pre y una ejecución operacional de p desde s_0 hasta s_1 :

$$\forall s_1. post \ s_1 \implies \exists s_0. pre \ s_0 \wedge runsto \ p \ s_0 \ s_1$$

En F^* , el teorema se codifica mediante la siguiente función con tipos dependientes:

```
let rec soundness
  (p : stmt) (pre : cond) (post : cond)
  (pf : isl_triple pre p post) (s1 : state { post s1 })
  : GTot (s0 : state { pre s0 } & runsto p s0 s1) (decreases pf)
```

La firma constituye la misma traducción de la caracterización sub-aproximada utilizada en el Capítulo 3:

- `s1 : state { post s1 }` representa un estado final perteneciente al resultado;
- `s0 : state { pre s0 }` representa el estado inicial testigo;
- `runsto p s0 s1` es la ejecución que conecta ambos estados.

La demostración de la función se realiza por inducción estructural sobre el árbol de prueba `pf`. Los constructores heredados de la IL siguen el argumento desarrollado en la Sección 3.4. Los nuevos casos corresponden a las operaciones de memoria y a la Regla del Marco.

En las reglas exitosas de memoria, las postcondiciones contienen valores existenciales que permiten reconstruir el estado previo. Por ejemplo, en `ISL_Free`, el estado final contiene una celda `Empty` y la postcondición garantiza la existencia del valor `v` que ocupaba originalmente la ubicación. La prueba extrae ese valor mediante una operación fantasma de elección clásica y reconstruye el *heap* inicial reemplazando `Empty` por `Full v`. Luego, usa `R_Free` para construir la ejecución operacional y `R_Ext` para ajustar posibles diferencias extensionales entre el *heap* calculado y el estado final recibido.

Los casos `ISL_FreeEr`, `ISL_LoadEr` e `ISL_StoreEr` son más directos. La postcondición ya contiene el recurso `points_to_empty` necesario para construir la regla operacional de error correspondiente. Por otro lado, los casos nulos utilizan la prueba de que la expresión evaluada es igual a 0 y construyen directamente `R_FreeNull`, `R_LoadNull` o `R_StoreNull`.

Si bien la mayoría de los casos axiomáticos reconstruyen el estado de forma directa, el caso de la Regla del Marco constituye el mayor desafío lógico de la formalización. Demostrar que `ISL_Frame` no introduce falsos positivos requiere coordinar el razonamiento deductivo de la lógica de separación con las pruebas de simulación semántica que se definieron en `r_frame`. Analicemos cómo el verificador reconstruye esta traza paso a paso.

Cuando la prueba fue construida mediante el marco, sabemos lógicamente que el estado final `s1` satisface la conjunción separadora `p_post ** fr`. Por la Definición 2.13 de la conjunción separadora, el *heap* final puede dividirse en dos fragmentos disjuntos:

$$hp_1 = h_1 \uplus h_2,$$

donde la postcondición local se satisface sobre h_1 , que de ahora en más llamaremos `h_local`, y la aserción del marco se satisface sobre h_2 , renombrado a `h_fr`.

En la prueba se define esta información mediante un predicado auxiliar:


```

let unfold p_two (h1 : heap) (h2 : heap) : prop =
  heaps_disjoint h1 h2 /\
  hp1 == heap_union h1 h2 /\
  p_post (st1, h1, m) /\
  fr (st1, h2, m)
in

```

La existencia de ambos *heaps* proviene de la definición de `sep_conj`. El comprobador SMT Z3 no puede instanciar computacionalmente esos sub-*heaps* por sí solo. Para materializarlos, se transforman los dos testigos existenciales en una tupla y se selecciona dicha tupla mediante `indefinite_description_ghost`.

Una vez aislado el fragmento local del *heap*, aplicamos la hipótesis inductiva de nuestro teorema.

```

let (|s0_local, r_local|) =
  soundness p_cmd (is_ok p_pre) p_post pf_p (st1, h_local, m) in

```

Esta llamada nos devuelve un estado inicial local y una traza válida entre dicho estado y la porción final `h_local`. El *heap* `h_local` es el fragmento que satisface la postcondición local proporcionada por la conjunción separadora. En esta parte de la prueba no se afirma todavía que dicho fragmento sea una huella mínima en el sentido de la Definición 2.22.

El último paso es elevar esa certeza local al contexto global, es decir, agregar `h_fr` al estado inicial local. La disjunción con el *heap* final se conoce por la partición de `s1`. La disjunción con el *heap* inicial se obtiene mediante:

```
lemma_runsto_disjoint h_fr r_local;
```

Luego, se usa la propiedad operacional del marco:

```
let r_global = r_frame r_local h_fr #() #() in
```

Esto produce una ejecución sobre los *heaps* globales. Finalmente, tenemos que la condición `independent_on_vars (modifies p) fr` y el lema que caracteriza las variables modificadas por `runsto` permiten demostrar que la aserción del marco también se satisfacía sobre el *store* inicial reconstruido. En conclusión, la verificación de la función `soundness` por parte de F^* mecaniza el Teorema 2.19. Luego:

$$\text{isl_triple } pre \ p \ post \implies \forall s_1 \in post. \exists s_0 \in pre. \text{runsto } p \ s_0 \ s_1$$

Por el Corolario 2.20, cuando `post` describe estados en modo `Er` y es satisfacible, existe una ejecución concreta del modelo formalizado que alcanza uno de esos estados de error.

4.6. El Teorema de la Huella

En la Sección 2.3 se presentó el Teorema de la Huella 2.23, que caracteriza la semántica operacional de un comando a partir de sus ejecuciones locales y de la extensión de estas con marcos disjuntos. El resultado dice:

$$\llbracket C \rrbracket_\epsilon = \text{frame}(\text{foot}(C)_\epsilon)$$

Esta igualdad reúne dos direcciones, la dirección local a global establece que una ejecución local sigue siendo válida al agregarle el marco. La dirección global a local establece que toda ejecución operacional puede descomponerse en una parte local y un marco disjunto.

La función `r_frame` ya mecaniza la primera parte para una ejecución cualquiera. Si `runsto p s0 s1` y un *heap* es disjunto de los *heaps* de `s0` y `s1`, entonces ese *heap* puede agregarse a ambos estados y se obtiene una nueva derivación de `runsto`. Sin embargo, esta propiedad por

sí sola no identifica qué parte de una ejecución es local ni demuestra que toda ejecución global admita una descomposición parecida.

Para obtener una caracterización completa se introducen dos nuevas relaciones. La primera de estas es **footprint**, que describe ejecuciones sobre fragmentos locales del *heap*. La segunda, **framed_footprint**, representa la clausura de esas ejecuciones con la incorporación de marcos disjuntos. Finalmente, se demuestra que **framed_footprint** y **runsto** contienen las mismas transiciones.

La relación de huellas

La relación **runsto** permite que los estados inicial y final tengan memoria adicional que no participa de la ejecución. Por ejemplo, una ejecución de **Load x e** puede ocurrir en un *heap* con varias celdas, aunque la instrucción solo contiene una única dirección. Para aislar el fragmento local utilizado por cada comando se define el siguiente tipo inductivo:

```
noeq type footprint : stmt -> state -> state -> Type0
```

Un habitante de **footprint p s0 s1** representa una ejecución local de **p** desde **s0** hasta **s1**. El tipo está indexado por el comando y por sus estados inicial y final. La diferencia con **runsto** es que sus constructores restringen los *heaps* a los fragmentos requeridos por cada operación.

Para representar los comandos que no acceden a memoria dinámica se define el *heap* vacío:

```
let empty_heap : heap = fun _ -> Unknown
```

Todas sus ubicaciones son **Unknown**, por lo que el fragmento no contiene ninguna celda activa ni negativa. Este *heap* se utiliza en las huellas de instrucciones como **Skip**, **Error**, **Assign**, **Nondet** y **Assume**, cuyos efectos solo involucran el *store* o el modo de terminación. Por ejemplo, la huella de **Skip** se codifica como:

```
| F_Skip : st : store ->
  footprint Skip (st, empty_heap, Ok) (st, empty_heap, Ok)
```

La instrucción no requiere recursos espaciales y conserva todos los componentes del estado local.

Para las operaciones de memoria se definen *heaps* formados por una única celda:

```
let singleton_empty_heap (l : loc) : heap =
  fun l' -> if l' = l then Empty else Unknown

let singleton_full_heap (l : loc) (v : value) : heap =
  fun l' -> if l' = l then Full v else Unknown
```

La primera describe exactamente una celda negativa, mientras que la segunda describe exactamente una celda activa en una dirección específica. Todas las demás direcciones quedan fuera del fragmento local.

Una lectura exitosa requiere únicamente la celda que será consultada:

```
| F_Load : st : store -> x : var -> e : expr -> l : loc { l != 0 } -> v : value ->
  #(squash (eval_expr' st e == 1)) ->
  footprint (Load x e) (st, singleton_full_heap l v, Ok)
  (override st x v, singleton_full_heap l v, Ok)
```

El *heap* inicial contiene una única celda activa en **l**. La lectura conserva esa celda y actualiza en el *store* el valor de **x**.

En cambio, una lectura sobre una ubicación previamente liberada requiere una celda negativa:

```
| F_LoadEr : st : store -> x : var -> e : expr -> l : loc { l != 0 } ->
  #(squash (eval_expr' st e == 1)) ->
  footprint (Load x e) (st, singleton_empty_heap l, Ok)
  (st, singleton_empty_heap l, Er)
```

En este caso, la operación no modifica el *heap*, pero cambia el modo de terminación. La celda negativa permanece en el estado final como justificación espacial del error.

Las reglas correspondientes a la escritura y a la liberación de memoria siguen el mismo patrón. Una liberación exitosa transforma un *heap* singleton positivo en uno negativo:

```
| F_Free : st : store -> e : expr -> l : loc { l != 0 } -> v : value ->
  #(squash (eval_expr' st e == 1)) ->
  footprint (Free e) (st, singleton_full_heap l v, 0k)
  (st, singleton_empty_heap l, 0k)
```

Una escritura exitosa conserva la dirección, pero reemplaza su contenido:

```
| F_Store : st : store -> e1 : expr -> e2 : expr -> l : loc { l != 0 } -> v :
  value ->
  #(squash (eval_expr' st e1 == 1)) ->
  footprint (Store e1 e2) (st, singleton_full_heap l v, 0k)
  (st, singleton_full_heap l (Nat (eval_expr' st e2)), 0k)
```

Los accesos a la dirección nula utilizan *empty_heap*, dado que el error se produce por el valor evaluado por la expresión y no requiere tener ninguna celda del *heap*.

De esta forma, los constructores básicos de *footprint* describen solo las celdas que intervienen en cada ejecución. Las demás ubicaciones no se eliminan de la semántica global, sino que luego serán incorporadas con un marco.

El caso más complejo de la definición es la composición secuencial. Las huellas de dos comandos consecutivos no tienen por qué usar los mismos recursos en el estado intermedio. El primer comando puede usar celdas que el segundo no necesita. Análogamente, el segundo puede requerir celdas que el primero no utilizó. Por lo tanto, no sería suficiente exigir que el *heap* final de la huella del primer comando coincida sintácticamente con el *heap* inicial de la huella del segundo.

Para representar esta situación, el constructor *F_Seq* divide la memoria del estado intermedio en tres componentes, h_{common} , h_p y h_q . El *heap* h_{common} contiene los recursos utilizados por los dos comandos. El *heap* h_p contiene los recursos exclusivos de *p*, mientras que h_q contiene los recursos exclusivos de *q*.

La estructura principal del constructor es:

```
| F_Seq : p : stmt -> q : stmt -> st0 : store -> stm : store ->
  st1 : store -> h_common : heap ->
  h_p : heap { heaps_disjoint h_common h_p } ->
  h_q : heap { heaps_disjoint h_common h_q /\ heaps_disjoint h_p h_q } ->
  h0 : heap { heaps_disjoint h0 h_q } ->
  h1 : heap { heaps_disjoint h1 h_p } ->
  m1 : term_mode ->
  footprint p (st0, h0, 0k) (stm, heap_union h_common h_p, 0k) ->
  footprint q (stm, heap_union h_common h_q, 0k) (st1, h1, m1) ->
  footprint (Seq p q) (st0, heap_union h0 h_q, 0k)
  (st1, heap_union h1 h_p, m1)
```

La huella local de *p* comienza con h_0 y termina usando los recursos comunes y los exclusivos del primer comando,

$$h_0 \xrightarrow{p} h_{common} \uplus h_p.$$

Sin embargo, para construir la ejecución completa de la secuencia, los recursos exclusivos de *q* deben estar presentes durante la ejecución de *p*, aunque este no los utilice. Por eso, h_q se agrega como marco,

$$h_0 \uplus h_q \xrightarrow{p} (h_{common} \uplus h_p) \uplus h_q.$$

De manera simétrica, la huella de *q* usa los recursos comunes y sus recursos exclusivos,

$$h_{common} \uplus h_q \xrightarrow{q} h_1.$$

Durante esta segunda ejecución, h_p permanece como marco,

$$(h_{common} \uplus h_q) \uplus h_p \xrightarrow{q} h_1 \uplus h_p.$$

Las condiciones de disjunción permiten demostrar que los dos estados intermedios globales son equivalentes,

$$(h_{common} \uplus h_p) \uplus h_q = (h_{common} \uplus h_q) \uplus h_p.$$

Por lo tanto, las dos ejecuciones pueden componerse mediante la regla operacional R_Seq .

La propagación de un error producido por el primer comando se representa con un constructor separado:

```
| F_SeqEr : p : stmt -> q : stmt ->
  s : state { s._3 == 0k } -> t : state { t._3 == Er } ->
  footprint p s t ->
  footprint (Seq p q) s t
```

Como el primer comando termina en error, el segundo no se ejecuta y no incorpora nuevos requisitos espaciales.

La relación incluye además el constructor F_Ext . Este constructor cierra la relación de huellas bajo equivalencia de estados.

```
| F_Ext : p : stmt -> s0 : state -> s1 : state -> footprint p s0 s1 ->
  s0' : state -> s1' : state ->
  squash (state_equiv s0 s0') ->
  squash (state_equiv s1 s1') ->
  footprint p s0' s1'
```

Dada una huella entre $s0$ y $s1$, permite reemplazar esos índices por estados $s0'$ y $s1'$ cuyos *stores*, *heaps* y modos de terminación coinciden punto a punto con los originales. F_Ext expresa para *footprint* lo que R_Ext expresa para *runsto*. La regla resulta necesaria porque los *stores* y los *heaps* se representan como funciones. Dos expresiones funcionales pueden coincidir para todos sus argumentos sin tener la misma definición para F^* . Esto ocurre especialmente al asociar y reordenar uniones de fragmentos en F_Seq .

Corrección operacional de las huellas

Una vez definida la relación local, el primer resultado consiste en demostrar que toda huella representa una ejecución operacional válida:

```
let rec footprint_sound (p : stmt) (s0 : state) (s1 : state)
  (f : footprint p s0 s1)
  : GTot (runsto p s0 s1) (decreases f)
```

La función establece

$$\text{footprint } ps_0s_1 \implies \text{runsto } ps_0s_1.$$

La demostración se hace por inducción estructural sobre f . Los casos primitivos se traducen al constructor correspondiente de *runsto*. En algunos casos, el *heap* final construido por la semántica operacional usa *override*, mientras que la huella usa un *heap* singleton definido directamente. Ambos *heaps* coinciden en todas sus ubicaciones, pero no están definidos de la misma manera. La prueba construye primero la regla operacional correspondiente y luego usa R_Ext para transportar la derivación al estado final indicado por la huella.

El caso F_Ext refleja la relación entre los dos niveles:

```
| F_Ext p s0 s1 fp s0' s1' e0 e1 ->
  let r = footprint_sound p s0 s1 fp in
  R_Ext r s0' s1' e0 e1
```

La hipótesis inductiva transforma la huella original en una ejecución de `runsto`. Luego, `R_Ext` transporta esa ejecución a los estados equivalentes.

El caso `F_Seq` requiere usar la propiedad operacional del marco. La hipótesis inductiva aplicada a la primera huella produce una ejecución local de `p`, a la cual se agrega `h_q` mediante `r_frame`. De manera análoga, la ejecución local de `q` se extiende con `h_p`. Después de ajustar los estados intermedios mediante la equivalencia, las dos derivaciones se componen con `R_Seq`.

Clausura por marcos

El Teorema de la Huella no relaciona directamente `footprint` con `runsto`, sino la semántica global con la clausura por marcos de las ejecuciones locales. Para representar dicha clausura se define:

```
noeq
type framed_footprint : stmt -> state -> state -> Type0 =
| FF_Frame : p : stmt -> s0 : state -> s1 : state ->
  h_fr : heap -> fp : footprint p s0 s1 ->
  d0 : squash (heaps_disjoint s0._2 h_fr) ->
  d1 : squash (heaps_disjoint s1._2 h_fr) ->
  framed_footprint p (s0._1, heap_union s0._2 h_fr, s0._3)
    (s1._1, heap_union s1._2 h_fr, s1._3)
| FF_Ext : p : stmt -> s0 : state -> s1 : state ->
  framed_footprint p s0 s1 -> s0' : state -> s1' : state ->
  squash (state_equiv s0 s0') -> squash (state_equiv s1 s1') ->
  framed_footprint p s0' s1'
```

`FF_Frame` recibe una huella local y agrega el mismo *heap* `h_fr` a sus estados inicial y final. Las dos premisas de disjunción garantizan que el marco no se superponga con los recursos locales de ninguno de los estados. Por otro lado, `FF_Ext` aplica al nivel de `framed_footprint` el mismo principio de equivalencia que `R_Ext` aplica a `runsto` y `F_Ext` a `footprint`.

La corrección operacional de la clausura se demuestra mediante:

```
let rec framed_footprint_sound (p : stmt) (s0 : state) (s1 : state)
  (ff : framed_footprint p s0 s1)
  : GTot (runsto p s0 s1) (decreases ff) =
match ff with
| FF_Frame p s0 s1 h_fr f_p _ _ ->
  let r_p = footprint_sound p s0 s1 f_p in
  r_frame r_p h_fr #() #()
| FF_Ext p s0 s1 f_p s0' s1' _ _ ->
  let r_p = framed_footprint_sound p s0 s1 f_p in
  R_Ext r_p s0' s1' _ _
```

En el caso `FF_Frame`, la función aplica primero `footprint_sound` para obtener una ejecución local y luego utiliza `r_frame` para agregar el marco. En el caso `FF_Ext`, aplica recursivamente la hipótesis inductiva y transporta la ejecución mediante `R_Ext`.

Por lo tanto,

$$\text{framed_footprint } p \ s_0 \ s_1 \implies \text{runsto } p \ s_0 \ s_1.$$

Esta función mecaniza la dirección local a global del Teorema 2.23,

$$\text{frame}(\text{footprint}(C)_\epsilon) \subseteq \llbracket C \rrbracket_\epsilon.$$

Toda ejecución local sigue siendo una ejecución operacional válida cuando se le agrega un marco compatible que permanece inalterado.

Descomposición de ejecuciones globales

Para completar la caracterización es necesario demostrar la dirección inversa. Dada una derivación operacional, debe ser posible separar los recursos utilizados localmente por el comando de aquellos que permanecen como marco. Esta propiedad se mecaniza con:

```
let rec runsto_decompose (#p : stmt) (#s0 #s1 : state) (r : runsto p s0 s1)
  : GTot (framed_footprint p s0 s1) (decreases r)
```

La función establece,

$$\text{runsto } p \ s_0 \ s_1 \implies \text{framed_footprint } p \ s_0 \ s_1$$

La demostración procede por inducción estructural sobre r .

Cuando la derivación operacional comienza con R_Ext , la hipótesis inductiva descompone la ejecución original y FF_Ext transporta el resultado:

```
| R_Ext #p #s0 #s1 r_base s0' s1' eq0 eq1 ->
  let ff = runsto_decompose r_base in
  FF_Ext p s0 s1 ff s0' s1' eq0 eq1
```

Para los comandos que no acceden al *heap*, la huella local usa `empty_heap` y el *heap* completo de la ejecución se coloca en el marco. Por ejemplo, una ejecución global

$$(st, h, Ok) \xrightarrow{\text{skip}} (st, h, Ok)$$

se descompone en la huella local

$$(st, \text{empty_heap}, Ok) \xrightarrow{\text{skip}} (st, \text{empty_heap}, Ok)$$

a la que se agrega el marco h con FF_Frame . Como `heap_union empty_heap h` puede no ser igual que h , se termina usando FF_Ext :

```
| R_Skip s ->
  let st = s._1 in
  let h = s._2 in
  let s_local = (st, empty_heap, Ok) in
  let s_framed = (st, heap_union empty_heap h, Ok) in
  let ff = FF_Frame Skip s_local s_local h (F_Skip st) () () in
  FF_Ext Skip s_framed s_framed ff s s () ()
```

En esta construcción, FF_Frame agrega los recursos globales, mientras que FF_Ext ajusta la representación funcional del estado resultante.

Las operaciones de memoria requieren separar del *heap* global la celda utilizada. Para ello se define:

```
let heap_without (h : heap) (l : loc) : heap =
  fun l' -> if l' = l then Unknown else h l'
```

Si una ejecución de liberación, lectura o escritura usa la dirección l , su huella local tiene el *heap* singleton correspondiente, mientras que `heap_without h l` se usa como marco.

Los lemas auxiliares demuestran que ambos fragmentos son disjuntos y que su unión reconstruye el *heap* original. Por ejemplo, si $h(l) = Full(v)$, entonces:

$$\text{singleton_full_heap } l \ v \perp \text{heap_without } h \ l \quad y$$

$$\text{singleton_full_heap } l \ v \oplus \text{heap_without } h \ l = h$$

El caso de una operación sobre una celda negativa se realiza de manera análoga.

Las reglas de elección no determinista y estrella de Kleene reutilizan la descomposición obtenida por la hipótesis inductiva y transforman únicamente la huella local. Para preservar las capas de marco y equivalencia se usa la función auxiliar `map_framed_footprint`. La estructura es la siguiente:

```

let rec map_framed_footprint (#p #q : stmt) (#s0 #s1 : state)
  (transform : (#s10 : state -> #s11 : state -> footprint p s10 s11 ->
    GTot (footprint q s10 s11)))
  (ff : framed_footprint p s0 s1)
  : GTot (framed_footprint q s0 s1) (decreases ff) =
match ff with
| FF_Frame _ s0_local s1_local h_fr fp d0 d1 ->
  let fp' = transform fp in
  FF_Frame q s0_local s1_local h_fr fp' d0 d1
| FF_Ext _ s0_base s1_base ff_base s0' s1' e0 e1 ->
  let ff_base' = map_framed_footprint transform ff_base in
  FF_Ext q s0_base s1_base ff_base' s0' s1' e0 e1

```

La función no agrega ni elimina marcos, solo transforma la huella local ubicada dentro de la estructura.

Si el primer comando de una secuencia termina en error, `framed_seq_er` transforma una huella enmarcada de `p` en una huella enmarcada de `Seq p q`. Como el estado final está en modo `Er`, no es necesario construir una huella para `q`.

El caso de `R_Seq` es la parte central de la prueba. Supongamos una ejecución operacional $s_0 \xrightarrow{p} t \xrightarrow{q} s_1$. Las llamadas recursivas producen descomposiciones de ambas sub-ejecuciones `framed_footprint p s0 t` y `framed_footprint q t s1`. Cada una divide el *heap* intermedio de manera diferente. La primera como

$$h_t = h_p^{local} \uplus h_p^{frame},$$

mientras que la segunda como

$$h_t = h_q^{local} \uplus h_q^{frame}.$$

Para construir `F_Seq` es necesario reconciliar estas dos particiones del mismo *heap*.

En primer lugar, una función auxiliar elimina las capas de `FF_Ext` y expone los componentes fundamentales de cada descomposición. Estos componentes serían los estados locales, el marco, la huella local y las equivalencias con los estado globales. La equivalencia entre las dos reconstrucciones del *heap* intermedio permite aplicar una partición cruzada, denominada *cross split*. Dadas dos descomposiciones,

$$h_1 \uplus h_2 = h_3 \uplus h_4,$$

la partición cruzada construye cuatro capas h_{13} , h_{14} , h_{23} y h_{24} , disjuntos dos a dos y tales que

$$h_1 = h_{13} \uplus h_{14},$$

$$h_2 = h_{23} \uplus h_{24},$$

$$h_3 = h_{13} \uplus h_{23},$$

$$h_4 = h_{14} \uplus h_{24}.$$

La estructura que almacena estos componentes se define mediante:

```

noeq
type cross_split_witness (h1 h2 h3 h4 : heap) = {
  h13 : heap; h14 : heap; h23 : heap; h24 : heap;
  pairwise_disjoint : squash (heaps_pairwise_disjoint h13 h14 h23 h24);
  split_h1 : squash (forall l. heap_merge h13 h14 l == h1 l);
  split_h2 : squash (forall l. heap_merge h23 h24 l == h2 l);
  split_h3 : squash (forall l. heap_merge h13 h23 l == h3 l);
  split_h4 : squash (forall l. heap_merge h14 h24 l == h4 l)
}

```

En la composición secuencial, estos cuatro componentes se interpretan como $h_{13} = h_{common}$, $h_{14} = h_p$, $h_{23} = h_q$ y $h_{24} = h_{ext}$. El *heap* h_{common} sería el de las huellas locales de ambos comandos. El *heap* h_p pertenece a la huella del primer comando, mientras que h_q a la del segundo. Por último, h_{ext} representa a los marcos de ambas sub-ejecuciones y puede usarse como marco exterior de toda la secuencia.

A partir de esta partición, las huellas obtenidas por las hipótesis inductivas se transportan con `F_Ext` a los estados que necesita `F_Seq`. Luego, se construye una huella local de `Seq p q` con `F_Seq`. El marco exterior se agrega con `FF_Frame`, y por último `FF_Ext` ajusta los estados reconstruidos a los estados originales de la derivación `R_Seq`.

La partición cruzada es necesaria porque el *heap* local del primer comando y el del segundo no tienen por qué ser iguales. El resultado identifica qué recursos son comunes, cuáles son exclusivos de cada comando y cuáles están fuera de la ejecución de la secuencia.

Resultado mecanizado

Las funciones `framed_footprint_sound` y `runsto_decompose` establecen las dos direcciones de la caracterización. Por lo tanto, se obtiene el siguiente resultado:

Teorema 4.2. *Para todo comando p y estados s_0 y s_1 ,*

$$\text{runsto } p \ s_0 \ s_1 \iff \text{framed_footprint } p \ s_0 \ s_1$$

En F^* , las dos expresiones se expresan como:

```
let footprint_theorem_sound (#p : stmt) (#s0 #s1 : state)
  (ff : framed_footprint p s0 s1) : GTot (runsto p s0 s1) =
  framed_footprint_sound p s0 s1 ff

let footprint_theorem_complete (#p : stmt) (#s0 #s1 : state)
  (r : runsto p s0 s1) : GTot (framed_footprint p s0 s1) =
  runsto_decompose r
```

La primera función demuestra que toda huella local extendida con un marco compatible constituye una ejecución operacional válida. La segunda demuestra que toda ejecución operacional puede descomponerse en una huella local y un marco que permanece inalterado.

En consecuencia, la clausura por marcos de la relación inductiva `footprint` coincide con `runsto`:

$$\llbracket C \rrbracket_\epsilon = \text{frame}(\text{foot}(C)_\epsilon).$$

Capítulo 5

Evaluación

“The purpose of computing is insight, not numbers.”

— Richard W. Hamming

En el Capítulo 4 pudimos construir, mecanizar y demostrar la solidez matemática de la ISL utilizando F^* . El objetivo de este capítulo es evaluar la expresividad de la formalización mediante una serie de casos de estudio representativos.

En particular, se estudian un error de uso de memoria después de su liberación, un puntero colgante generado por una reasignación, un bucle con elecciones no deterministas y el recorrido de una lista enlazada. Estos casos muestran que la formalización puede expresar y verificar derivaciones no triviales.

5.1. Detección de errores espaciales básicos (*Use After Free*)

Para el primer caso de evaluación, analizaremos la capacidad del sistema para detectar el error espacial por excelencia en lenguajes de bajo nivel, el *use after free*. Este error ocurre cuando un programa intenta leer o escribir en una dirección de memoria después de que esta fue liberada y dejó de estar activa para el programa.

En la formalización, una ubicación liberada no desaparece del *heap*. Como se mencionó en el Capítulo 4, la operación **Free** transforma una celda **Full** v en una celda **Empty**. Este recurso negativo permite conservar la información de que la dirección fue liberada y usarla después para justificar una ejecución errónea.

Consideramos el siguiente programa a verificar escrito en nuestra sintaxis:

```
let prog_uaf =  
  (Alloc "x") 'Seq'  
  ((Free (Var "x")) 'Seq'  
   (Store (Var "x") (Const 1)))
```

La presunción inicial es `is_ok emp`. Esta aserción describe un estado en el modo exitoso cuyo fragmento local del *heap* no contiene ubicaciones conocidas. La propiedad que se busca verificar tiene la forma

$$\vdash_{\text{ISL}} [\text{is_ok } \text{emp}] \text{ prog_uaf } [\text{prog_uaf_err}].$$

En este caso, la postcondición final describe un estado con el modo erróneo en el que la dirección almacenada en `x` continúa representada con un *heap* negativo.

La primera instrucción que se invoca es `ISL_Alloc1`, la cual reserva una dirección de memoria nueva respecto del fragmento local. Luego de su ejecución, la aserción final del comando garantiza que la memoria aloja un valor válido en el canal de éxito:


```

let post_alloc (s : state) : prop =
  exists (l : loc) (v : value).
    s._3 == 0k /\ s._1 "x" == Loc l /\
    l != 0 /\ points_to l v s /\
    (exists x_old. emp (override s._1 "x" x_old, override s._2 l Unknown, 0k))

```

Esta postcondición garantiza que la variable `x` contiene una dirección no nula y que el fragmento local tiene una celda activa en esa ubicación. El cuantificador sobre `x_old` conserva la información necesaria para reconstruir el *store* anterior a la asignación.

El segundo paso del programa es la liberación de la memoria apuntada por `x`. Al evaluar el axioma `ISL_Free`, la semántica consume el recurso activo y lo transforma en una aserción negativa. El estado resultante refleja la transición:

```

let post_free (s : state) : prop =
  exists (v : value).
    s._3 == 0k /\ points_to_empty (eval_expr s (Var "x")) s /\
    post_alloc (s._1, override s._2 (eval_expr s (Var "x")) (Full v), 0k)

```

En este punto, la variable `x` sigue apuntando a la misma dirección física. La diferencia está en el *heap*, ya que la celda ya no se describe como `Full v`, sino como `Empty`.

El fallo ocurre en la tercera instrucción, cuando el programa intenta realizar una mutación de memoria sobre el puntero invalidado. Entre las reglas disponibles para la instrucción de escritura está `ISL_StoreEr`, la cual exige tener estrictamente el recurso `points_to_empty`, nuestro estado actual. Por lo tanto, la postcondición final se define como:

```

let post_uaf_err (s : state) : prop =
  s._3 == Er /\ points_to_empty (eval_expr s (Var "x")) s /\
  post_free (s._1, s._2, 0k)

```

La ejecución cambia al modo `Er`, pero conserva el *heap* negativo que justifica el error. De esta manera, la derivación no solo afirma que hubo un fallo, sino que contiene la información espacial necesaria para mostrar que se debe a una escritura sobre una ubicación liberada.

La construcción de esta demostración en F^* se realiza componiendo los axiomas de nuestra lógica mediante constructores secuenciales y la regla de consecuencia. El árbol de prueba completo para este programa se codifica de la siguiente manera:

```

let proof_uaf : isl_triple (is_ok emp) prog_uaf post_uaf_err =
  let p_alloc_raw = ISL_Alloc1 #emp "x" in
  let p_alloc = ISL_Consequence emp post_alloc p_alloc_raw () () in
  let p_free_raw = ISL_Free #post_alloc (Var "x") in
  let p_free = ISL_Consequence post_alloc post_free p_free_raw () () in
  let p_store = ISL_StoreEr #post_free (Var "x") (Const 1) in
  let p_seq1 = ISL_Seq p_free p_store in
  let p_seq2 = ISL_Seq p_alloc p_seq1 in
  ISL_Consequence emp post_uaf_err p_seq2 () ()

```

Las reglas `ISL_Alloc1`, `ISL_Free` e `ISL_StoreEr` devuelven las derivaciones correspondientes a las operaciones primitivas. `ISL_Seq` las compone de acuerdo con la estructura del programa. Las aplicaciones de `ISL_Consequence` adaptan las postcondiciones generadas directamente por las reglas a las aserciones intermedias elegidas para este caso.

La aceptación de `proof_uaf` por F^* demuestra que la tripleta es derivable. Por el Teorema de Soundness 4.1, todo estado que satisface `post_uaf_err` tiene un estado inicial que satisface `is_ok emp` y una ejecución `prog_uaf` que lo alcanza. Como `prog_uaf_err` describe estados en modo de error y conserva la celda negativa, el caso muestra que la formalización puede representar y verificar una ejecución concreta de *use after free* mediante un estado con respaldo operacional.

5.2. Reasignación de memoria y punteros colgantes

Un desafío en el análisis estático de software de bajo nivel, como C o C++, es el manejo de estructuras de datos dinámicas, tales como los vectores elásticos (`std::vector`). Cuando un programa agrega un elemento a una colección mediante una función como `push_back`, la biblioteca subyacente decide en tiempo de ejecución si posee capacidad suficiente o si debe reasignar el bloque de memoria para acomodar los nuevos elementos. Esta reasignación implica alojar un nuevo bloque y liberar la memoria original.

El problema surge si un cliente del código conserva un puntero hacia un elemento de la estructura original y luego invoca la función de agregado, ya que corre el riesgo de que la memoria sea reasignada. En ese escenario, su puntero local se convierte en un puntero colgante. Este fue un caso de estudio motivacional para el diseño de la ISL original.

El siguiente fragmento de código abstrae conceptualmente este comportamiento en un lenguaje similar a C:

```
int* x = *v;
push_back(v); // Puede no hacer nada, o hacer free(*v) y reasignar
*x = 88;      // Posible uso de puntero colgante
```

En nuestra formalización F^* , modelamos este comportamiento mediante la definición del comando `client`:

```
let client (ptr : var) : stmt =
  (Load "x" (Var ptr)) 'Seq'
  ((push_back ptr) 'Seq' (Store (Var "x") (Const 88)))
```

La secuencia de operaciones del cliente se divide en tres etapas:

1. **Captura de la referencia:** El cliente accede a la estructura original para guardar un puntero directo a su memoria activa mediante la instrucción de lectura `Load`.
2. **Interacción con la biblioteca:** El programa cliente invoca la función de agregado llamando a `push_back`. Esta función de biblioteca encapsula un comportamiento no determinista interno, por lo que el cliente desconoce si la estructura mantendrá su bloque de memoria original o si será movida y reasignada en un bloque nuevo:

```
let push_back (ptr : var) : stmt =
  Choice
    ((Load "y" (Var ptr)) 'Seq'
     ((Free (Var "y"))) 'Seq'
     ((Alloc "y") 'Seq'
      (Store (Var ptr) (Var "y"))))
  Skip
```

3. **Mutación sobre el alias:** Asumiendo erróneamente que su puntero local sigue siendo válido, el cliente intenta realizar una mutación sobre él mediante la instrucción de escritura `Store`.

La primera lectura almacena en `x` la dirección original de la estructura. La postcondición correspondiente registra el valor cargado en `x`, la celda que contiene la referencia de la estructura y la memoria activa a la que apunta dicha referencia.

Para demostrar el error no es necesario analizar las dos ramas de `push_back`. La interpretación sub-aproximada permite seleccionar la rama que conduce al estado de interés. En este caso, la derivación usa `p_push_back_raw = ISL_ChoiceL p_seq_pb1`. Esto no afirma que la rama derecha sea imposible, sino que la rama izquierda tiene una ejecución válida que alcanza la postcondición buscada.

La postcondición de esa rama describe que la estructura apunta a una nueva ubicación. El recurso negativo correspondiente a la dirección anterior se conserva dentro de una cadena de aserciones históricas producidas por las reglas de liberación y asignación. Por ejemplo, la aserción posterior a la liberación contiene:

```
let post_free_y (ptr : var) (s : state) : prop =
  exists (v : value).
    s._3 == 0k /\
    points_to_empty (eval_expr s (Var "y")) s /\
    post_load_y ptr
    (s._1, override s._2 (eval_expr s (Var "y")) (Full v), s._3)
```

La dirección histórica está representada por `points_to_empty`.

Después de la reasignación, el cliente intenta una mutación. La variable `x` sigue almacenando la dirección original. Esta forma encaja en el axioma de fallo `ISL_StoreEr`. En nuestro código, la postcondición de error tiene la forma:

```
let post_err (ptr : var) (s : state) : prop =
  s._3 == Er /\
  points_to_empty (eval_expr s (Var "x")) s /\
  post_store_v ptr (s._1, s._2, 0k)
```

Esta postcondición describe una ejecución que termina en el modo de error y conserva la evidencia espacial de que la dirección utilizada mediante `x` fue liberada.

Desde el comienzo se mencionó que la prueba iba a abarcar solo una de las ramas posibles de ejecución. En particular, la aplicación de la regla de elección es:

```
let p_push_back_raw = ISL_ChoiceL p_seq_pb1 in
let p_push_back = ISL_Consequence (post_load_x ptr) (post_store_v ptr)
  p_push_back_raw () () in
```

Luego, la derivación de `push_back` se compone con la lectura inicial y la escritura final.

La rama `Skip`, en la que el bloque no se reasigna, no forma parte de este árbol de prueba. Esto no implica que se la considere errónea o imposible. La sub-aproximación permite omitirla porque el objetivo consiste en justificar la existencia de una ejecución que produzca un fallo.

Este caso muestra que la formalización puede relacionar un error espacial con una modificación intermedia realizada a través de otro alias. La celda negativa conserva la historia del bloque original, mientras que el no determinismo permite seleccionar la ejecución de reasignación.

5.3. No determinismo y razonamiento paramétrico sobre bucles

A lo largo del desarrollo de software, la presencia de bucles combinados con comportamientos no deterministas supone una barrera para las técnicas de verificación tradicionales. Para comprender los límites prácticos de dichas metodologías frente a la sub-aproximación, evaluaremos un programa semi-realista con una alta cantidad de iteraciones y decisiones no deterministas. El caso busca mostrar dos propiedades del sistema deductivo:

1. una derivación sub-aproximada puede seleccionar un único camino entre varias alternativas (en la Sección 5.2 se vio en menor magnitud);
2. la Regla de la Variante de Kleene permite representar paramétricamente una familia de iteraciones sin construir una derivación distinta para cada una.

Consideremos el siguiente programa:

```
n := 1000000;
i, j := 0;
```

```

while (i < n){
  i++;
  if (random()) j++;
}
assert (j != n);

```

El bucle ejecuta un millón de veces. En cada iteración, *i* se incrementa siempre, mientras que el incremento de *j* depende de una decisión no determinista. Si la rama de incremento se selecciona en todas las iteraciones, al finalizar se cumple que tanto *i* como *j* son iguales a un millón. En ese estado, la aserción *j != n* falla.

En la semántica formalizada, la llamada `random()` se representa con un `Choice`. No se modelan probabilidades, independencia estadística ni distribuciones. Si lo interpretamos bajo la visión de un programa ejecutable en la que cada decisión es independiente y equiprobable, la probabilidad de incrementar *j* en el millón de iteraciones sería de $2^{-1000000}$. Este cálculo no interviene en la demostración mecanizada. La lógica solo necesita establecer que la secuencia de elecciones existe.

El primer paso es modelar la aleatoriedad de forma que podamos controlarla lógicamente. En nuestra sintaxis abstracta, la condición se modela mediante el operador de elección no determinista `Choice`, codificado con el operador `|||`:

```
Assign "j" (Plus (Var "j") (Const 1)) ||| Skip
```

Para construir la ejecución que lleva al error, la derivación usa `ISL_ChoiceL` y selecciona la rama de incremento en el cuerpo del bucle. En una teoría de sobre-aproximación, el axioma de esta instrucción nos obligaría a arrastrar una disyunción en la postcondición, ramificando el estado lógico en cada iteración.

A pesar de poder elegir una sola rama, construir un millón de aplicaciones de la regla de Kleene produciría una derivación enorme. Para evitar ese desenrollado se usa `ISL_KleeneVariant`.

La familia de aserciones se define como sigue:

```

let variant (k : nat) : cond =
  fun (st, _, m) ->
    k <= 1000000 /\ st "n" == Nat 1000000 /\
    st "i" == Nat k /\ st "j" == Nat k /\ m == Ok

```

Este predicado asocia el número de iteración lógica *k* con los valores concretos del *store*. Afirma que, después de *k* iteraciones del camino seleccionado, tanto *i* como *j* son iguales a *k*.

Para aplicar `ISL_KleeneVariant` debe demostrarse paramétricamente que el cuerpo transforma `variant k` en `variant k+1`. La demostración de este paso se hace para un *k* arbitrario. La rama izquierda de `Choice` incrementa *j*, mientras que las asignaciones restantes incrementan *i*. Las obligaciones aritméticas locales son verificadas por el *solver*. Una vez demostrado el paso general, la regla permite obtener estados correspondientes a una cantidad finita de iteraciones sin construir manualmente una derivación diferente para cada valor del índice.

La instrucción de aserción se codifica como una elección entre el camino de error y el exitoso:

```

let assert_stmt =
  Seq (Assume (Eq (Var "j") (Var "n"))) Error |||
  Assume (enot (Eq (Var "j") (Var "n")))

```

La rama izquierda representa el caso de *j* y *n* iguales, donde el `Assume` permite continuar y luego `Error` cambia el modo de terminación. La rama derecha representa el caso en que *j* y *n* son distintos.

La variante construida para el camino seleccionado permite alcanzar el estado en el que tanto *i* como *j* como *n* son iguales a un millón. Por lo tanto, la igualdad evaluada por el primer `Assume` es verdadera. La derivación vuelve a usar `ISL_ChoiceL` para seleccionar la rama que lleva a `Error`.

La postcondición final es:

```
let post_er_bug : cond =
  fun (st, _, m) ->
    st "n" == Nat 1000000 /\ st "i" == Nat 1000000 /\
    st "j" == Nat 1000000 /\ m == Er
```

Un análisis que enumera todas las secuencias de decisiones podría tener hasta $2^{1000000}$ combinaciones. La derivación de este caso no enumera dichas combinaciones. Selecciona una rama concreta y usa una familia de aserciones para representar las iteraciones.

5.4. Recorrido de una lista enlazada

El último caso combina el razonamiento iterativo de la sección anterior con estructuras de datos almacenadas en memoria dinámica. Para este escenario, formalizamos un programa que recorre una lista enlazada representada mediante nodos de dos celdas, acumulando la suma de sus elementos en la variable `sum` y contando su longitud en la variable `len`. Al finalizar, el programa verifica mediante una aserción que la suma total sea distinta a la longitud calculada. Si la suma es igual, la aserción falla y el programa aborta.

Conceptualmente, el código bajo análisis se describe de la siguiente manera:

```
sum = 0;
len = 0;
while (ptr != 0) {
  v = *ptr;
  sum += v;
  len++;
  ptr = *(ptr + 1);
}
assert(sum != len);
```

Para este caso, modelamos una lista enlazada de longitud arbitraria `n_target`. Aunque en este ejemplo se fija `n_target` en diez, la demostración es paramétrica en la longitud de la lista y no requiere desenrollar una iteración por cada nodo. Además, se considera como precondition una lista cuyos elementos son todos iguales a 1. Esta condición es suficiente para garantizar que, al finalizar el recorrido, la suma de los elementos coincide con la longitud de la lista y la aserción falla. No es la única configuración posible que satisface dicha igualdad, pero permite expresar el argumento mediante un predicado espacial paramétrico simple.

En lenguajes de bajo nivel como C, un nodo de una lista enlazada se representa típicamente como una estructura que agrupa datos y referencias. En nuestra formalización, modelamos esta estructura desarmándola en sus componentes de memoria fundamentales. Un nodo ocupa dos ubicaciones de memoria adyacentes:

1. la dirección base `curr_ptr`, que almacena el valor numérico del nodo y
2. la dirección consecutiva `curr_ptr + 1`, que almacena el puntero hacia el siguiente nodo de la estructura.

Para describir una lista con una longitud arbitraria se definen los predicados `list_seg` y `prefix_seg`. El caso de `list_seg start finish xs` describe un segmento de lista que empieza en `start`, termina en `finish` y tiene la secuencia de valores `xs`. El caso vacío representa un segmento sin nodos. El caso inductivo describe las dos celdas del primer nodo y el segmento restante con la conjunción separadora. Por otro lado, `prefix_seg start current k` describe la porción inicial de una lista que contiene los primeros `k` nodos y termina inmediatamente antes de `current`.

Estos predicados permiten expresar la estructura completa sin enumerar cada dirección de forma individual. La conjunción separadora garantiza que las celdas de nodos diferentes pertenezcan a fragmentos disjuntos.

Durante cualquier iteración k , el programa debe mantener referencias tanto a lo que ya sumó como a lo que falta sumar. En este momento el *heap* se divide en cuatro partes,

1. el prefijo ya visitado;
2. la celda que tiene el valor del nodo actual;
3. la celda que tiene el puntero al nodo siguiente;
4. el sufijo todavía no visitado.

Luego, definimos la aserción de estado intermedio de la siguiente manera:

```
let post_len_shape (start_ptr curr_ptr next_ptr : loc) (k : nat) : cond =
  fun s ->
    k < n_target /\ s._3 == 0k /\ s._1 "sum" == Nat (k + 1) /\
    s._1 "len" == Nat (k + 1) /\ s._1 "v" == Nat 1 /\
    s._1 "ptr" == Loc curr_ptr /\ points_to curr_ptr (Nat 1) s /\
    (prefix_seg start_ptr curr_ptr k **
     points_to (curr_ptr + 1) (Loc next_ptr) **
     list_seg next_ptr 0 (ones (n_target - k - 1))) s
```

Los cuatro componentes se encuentran dentro de una única conjunción separadora. Por lo tanto, el *heap* completo debe poder dividirse en cuatro fragmentos disjuntos:

$$h = h_{\text{prefix}} \uplus h_{\text{value}} \uplus h_{\text{next}} \uplus h_{\text{suffix}}.$$

El prefijo tiene la parte ya recorrida de la lista. La celda *curr_ptr* tiene el valor 1, que acaba de cargarse en *v*. La celda *curr_ptr + 1* guarda la dirección *next_ptr*. Por último, el sufijo describe los nodos pendientes. Esto se puede observar en la Figura 5.1.

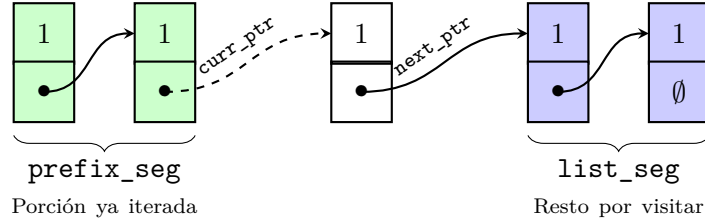


Figura 5.1: Partición espacial de la lista durante la iteración k .

La definición de la conjunción separadora exige que estos *heaps* sean disjuntos. Al avanzar a la iteración $k + 1$, se extrae el nodo de la cabeza del sufijo, se evalúa, y se empaqueta lógicamente dentro del prefijo. De esta manera, el razonamiento sobre el nodo actual no puede usar accidentalmente una celda que pertenezca al prefijo o al sufijo.

Si el programa original fuese inicializado con una memoria arbitraria, el verificador deductivo no podría afirmar la relación final entre la suma y la longitud. En nuestra demostración, inicializamos la prueba exigiendo en la precondition que el *heap* contenga el predicado *list_seg* poblado exactamente con una lista de unos:

```
let pre_init (start_ptr : loc) : cond =
  fun s -> let xs = ones n_target in
    (exists (curr_ptr : loc).
     s._3 == 0k /\ s._1 "ptr" == Loc curr_ptr /\
     (prefix_seg start_ptr curr_ptr 0 ** list_seg curr_ptr 0 xs) s)
```

Durante el cuerpo del bucle, la regla de lectura extrae el valor v de la celda actual. Gracias al particionamiento de la huella espacial que se definió previamente, el axioma garantiza lógicamente que el valor leído de la variable es 1. Luego, el programa incrementa las variables acumuladoras. En nuestra mecanización, hemos formalizado este avance lógico mediante lemas de correctitud locales. En particular, el avance de `sum` se respalda con el siguiente lema auxiliar que reconstruye el *store* anterior a la asignación:

```
let lemma_prove_sum_step (start_ptr : loc) (k : nat) (s : state)
  : Lemma (requires (post_sum start_ptr k s))
    (ensures (is_ok (fun s0 -> exists (st_init : var -> value).
      post_load_v start_ptr k (st_init, s0._2, s0._3) /\
      s._1 "sum" == Nat (eval_expr (st_init, s0._2, s0._3)
        (Plus (Var "sum") (Var "v")))) /\
      (forall (y : var). y <> "sum" ==> s._1 y == st_init y)) s))
```

El lema correspondiente a `len` sigue el mismo patrón.

Estas demostraciones no ejecutan el programa para cada valor de k . Establecen algebraicamente que, si antes de la asignación se cumple `sum = len = k`, entonces después de las actualizaciones se cumple `sum = len = k + 1`.

A partir de este punto, el paso entre una iteración y la siguiente modifica la partición lógica del *heap*. El nodo actual deja de formar parte del sufijo y pasa a integrarse al prefijo. El nuevo valor de `ptr` se obtiene de la celda `curr_ptr + 1`, por lo que la siguiente iteración empieza en `next_ptr`.

Al igual que en la Sección 5.3, el recorrido se demuestra con `ISL_KleeneVariant`. En este caso, la familia de aserciones relaciona el índice k con la longitud del prefijo ya visitado, los valores de `sum` y `len`, la posición actual de `ptr` y la estructura del sufijo restante. El paso inductivo demuestra que una ejecución del cuerpo transforma la aserción correspondiente a k en la correspondiente a $k + 1$. Cuando $k = n_target$, el sufijo es vacío y `ptr` contiene la dirección nula. Además, `sum = n_target = len`.

La aserción se representa como:

```
let assert_stmt =
  Seq (Assume (Eq (Var "sum") (Var "len"))) Error |||
  Assume (enot (Eq (Var "sum") (Var "len")))
```

Como la variante final establece `sum = len`, la derivación selecciona la rama izquierda con `ISL_ChoiceL`. El `Assume` conserva el estado y `Error` cambia el modo de terminación a `Er`.

Este caso de estudio combina los principales componentes espaciales e iterativos de la formalización. Estos son los *heaps* con muchas celdas, la conjunción separadora, los predicados recursivos, la Regla del Marco, lectura de memoria, asignaciones sobre el *store*, una variante paramétrica de Kleene y la selección de una rama de error. La derivación muestra que el sistema puede razonar sobre una estructura enlazada de longitud paramétrica sin escribir manualmente una prueba distinta para cada nodo.

5.5. Resumen de los resultados

Los cuatro casos muestran distintas capacidades del núcleo lógico. El primero valida el mecanismo básico de *heaps* negativos, mostrando que la liberación de una celda conserva la información necesaria para justificar un acceso posterior erróneo.

El segundo muestra que esta información puede propagarse a través de una secuencia más compleja que involucra alias y una llamada no determinista. La derivación selecciona el comportamiento de reasignación y conserva la dirección histórica que usa después el cliente.

El tercer caso muestra que la sub-aproximación permite concentrarse en una única secuencia de decisiones. La Regla de la Variante de Kleene permite expresar el efecto de una cantidad

arbitraria de iteraciones con una familia de aserciones, sin materializar un árbol diferente para cada valor concreto.

Finalmente, el recorrido de la lista enlazada combina el razonamiento paramétrico con predicados espaciales recursivos. La conjunción separadora permite dividir el *heap* entre la parte ya visitada, el nodo actual y el resto de la estructura.

Esta evaluación no establece completitud respecto de todos los programas posibles, no mide el rendimiento de la verificación y no demuestra que las aserciones puedan sintetizarse automáticamente. Su alcance consiste en validar que las construcciones desarrolladas en el Capítulo 4 puedan usarse para demostrar la alcanzabilidad de errores no triviales.

Capítulo 6

Conclusiones

“We can only see a short distance ahead, but we can see plenty there that needs to be done.”

— Alan M. Turing

6.1. Resumen de aportes

En esta tesina se mecanizaron la Lógica de Incorrectitud y su extensión con razonamiento espacial, la Lógica de Incorrectitud de Separación, utilizando el asistente de pruebas F^* . El desarrollo permite representar programas, ejecuciones y derivaciones deductivas dentro del sistema de tipos, y verificar formalmente que los resultados obtenidos mediante las reglas de ambas lógicas se encuentran respaldados por la semántica operacional.

En una primera etapa se formalizó la IL sobre un lenguaje imperativo simplificado. Se definió una semántica operacional con el tipo inductivo `runsto` y se representaron las reglas deductivas mediante el tipo inductivo indexado `il_triple`. De esta manera, la semántica del lenguaje y la derivabilidad dentro de la lógica quedaron expresadas como relaciones diferentes. Sobre estas definiciones se mecanizó el Teorema de Soundness de la IL. Este resultado establece que, para cada estado perteneciente a la postcondición de una tripleta derivable, existe un estado inicial que satisface la presunción y una ejecución operacional del comando que conecta los dos estados.

La formalización de la IL incluye las reglas correspondientes a asignación, secuencia, elección no determinista, suposición, error e iteración. También se representó una variante paramétrica de la regla de Kleene, que permite razonar sobre una cantidad finita pero arbitraria de iteraciones con una familia de aserciones indexada por números naturales.

En una segunda etapa se extendió el desarrollo para formalizar la ISL. Para ello se incorporó un *heap* como componente independiente del estado y se definieron tres clases de celdas, `Full`, `Empty` y `Unknown`. Las celdas `Full` representan ubicaciones activas, las celdas `Empty` representan ubicaciones conocidas como liberadas y `Unknown` indica que una ubicación no forma parte del fragmento local en el que se está trabajando. Esta distinción permite conservar información sobre las ubicaciones liberadas y utilizarla para después justificar accesos erróneos. Sobre este modelo se definieron las aserciones espaciales `emp`, `points_to` y `points_to_empty`, junto con la conjunción separadora. También se extendieron la sintaxis y la semántica operacional con instrucciones de reserva, lectura, escritura y liberación de memoria.

Las reglas deductivas de la ISL se representaron con el tipo inductivo `isl_triple`. Además de las reglas para las operaciones primitivas, se formalizó la Regla del Marco, con la condición de independencia entre la aserción usada como marco y las variables modificadas por el comando.

Sobre esta formalización se mecanizó el Teorema de Soundness de la ISL. Al igual que en el caso anterior, el resultado relaciona cada estado final descrito por su derivación con un estado inicial y una ejecución operacional concreta. La demostración del caso de la Regla del

Marco requirió probar que una ejecución local puede extenderse con un *heap* disjunto que no es modificado, propiedad que expresa la función `r_frame`.

También se formalizó una relación inductiva denominada `footprint`, cuyos constructores describen ejecuciones sobre los fragmentos locales de memoria requeridos por cada comando. A partir de esta definición se introdujo la relación `framed_footprint`, que representa la clausura de las huellas locales con marcos disjuntos y equivalencias. Sobre estas relaciones se demostró:

$$\text{framed_footprint } p \ s_0 \ s_1 \iff \text{runsto } p \ s_0 \ s_1.$$

La implicación de izquierda a derecha demuestra que toda huella extendida con un marco compatible constituye una ejecución operacional válida. El otro lado establece que toda ejecución operacional puede descomponerse en una huella local y un marco que no es modificado.

Finalmente, la formalización fue evaluada con cuatro casos de estudio. Se construyeron derivaciones para un error *use after free*, un puntero colgante producido por una reasignación no determinista, una ejecución con una gran cantidad de iteraciones y decisiones no deterministas, y el recorrido de una lista enlazada.

El desarrollo obtenido constituye un núcleo lógico mecanizado y la formalización garantiza el respaldo operacional de las derivaciones construidas.

6.2. Trabajo relacionado

Existen distintos trabajos sobre lógicas de programas, análisis estático y razonamiento sobre la presencia de errores. Los presentamos a continuación, comparándolos con el desarrollo realizado en esta tesina.

Mecanización de la HL para código máquina

Myreen and Gordon [14] presentan un framework mecanizado de estilo Hoare para razonar sobre programas escritos en código máquina. La lógica se construye sobre una semántica operacional que representa características propias de máquinas reales, como la memoria finita, la coexistencia de código y datos en un mismo espacio de direcciones, los registros de estado y el riesgo de modificar registros especiales, entre ellos el contador de programa, el puntero de pila y el registro de retorno. El desarrollo fue formalizado en lógica de orden superior y mecanizado en HOL4 [8].

El objetivo de ese trabajo es permitir la especificación y verificación de propiedades funcionales y de uso de recursos directamente sobre programas de bajo nivel. Aunque el modelo conserva detalles de la máquina, las especificaciones se expresan con una especificación flexible del estado inspirada en el razonamiento local de la SL.

El uso de una semántica operacional explícita y de un asistente de pruebas para justificar las reglas de razonamiento es un enfoque que comparte ese trabajo con la tesina actual. En los dos casos, las propiedades del sistema lógico no dependen solo de una argumentación informal, sino que se encuentran respaldadas por una mecanización.

La diferencia principal se encuentra en la interpretación de las tripletas. El trabajo de Myreen y Gordon se orienta al razonamiento de correctitud. En esta tesina, en cambio, las tripletas de IL e ISL tienen una interpretación sub-aproximada y se usan para demostrar estados descritos por la postcondición son alcanzables.

También difiere el nivel de abstracción. Myreen y Gordon trabajan directamente con código máquina y tienen que representar varios detalles de la arquitectura. La presente formalización usa un lenguaje imperativo reducido, lo que permite concentrar el desarrollo en la estructura de la IL y de la ISL y las demostraciones de sus propiedades metateóricas.

SL para la semántica de paso pequeño de Cminor

Appel and Blazy [1] presentan una formalización de la SL para Cminor, un lenguaje imperativo intermedio usado dentro de la infraestructura de CompCert. El trabajo redefine Cminor para facilitar el razonamiento con la HL y da una semántica operacional de paso pequeño, en lugar de solo usar la semántica de paso grande del compilador. Esta elección se encuentra motivada por la posibilidad de extender luego el lenguaje con concurrencia.

El desarrollo incluye una demostración mecanizada en Coq [2] de la solidez de la SL respecto de esa semántica. Los autores presentan el resultado como la primera demostración mecanizada de gran escala de una SL respecto de una semántica de paso pequeño. Además, relacionan la nueva semántica con la semántica usada por el compilador, con el objetivo de conservar las garantías de corrección de punto a punto.

Este trabajo es cercano a esta tesina por su metodología. En los dos desarrollos se define una semántica operacional, se formaliza un sistema de razonamiento espacial y se demuestra que las reglas de la lógica son sólidas respecto de dicha semántica. También comparten el uso de una representación explícita del *heap*, aserciones espaciales y razonamiento local con marcos.

Sin embargo, las lógicas formalizadas tienen interpretaciones diferentes. Appel y Blazy usan la SL para demostrar propiedades de correctitud y seguridad de programas Cminor. Esta tesina formaliza la ISL, cuyo objetivo es demostrar la existencia de ejecuciones que alcanzan determinadas postcondiciones, incluidos estados de error. Esta diferencia se refleja en los teoremas de *soundness*. En el razonamiento de correctitud, la prueba debe garantizar que toda ejecución considerada respete la postcondición. En la interpretación sub-aproximada, debe demostrarse que cada estado descrito por la postcondición tiene al menos una ejecución que lo alcanza desde algún estado que satisface la presunción.

Otra diferencia es la incorporación de *heaps* negativos. La formalización desarrollada en esta tesina distingue entre celdas activas, celdas liberadas y direcciones que no pertenecen al fragmento local. Esta representación permite justificar accesos erróneos a memoria liberada, un fin diferente al perseguido por la SL de Cminor.

IsaBIL: verificación de correctitud e incorrectitud de binarios

Griffin et al. [9] presentan IsaBIL, un framework desarrollado en Isabelle/HOL para verificar propiedades de programas binarios. La herramienta usa *Binary Analysis Platform* (BAP) [3] para traducir los binarios al lenguaje intermedio BIL y genera un entorno dentro de Isabelle/HOL en el que pueden construirse pruebas basadas tanto en HL como IL. IsaBIL incluye una formalización de la sintaxis y la semántica operacional de BIL. Las lógicas de correctitud e incorrectitud se definen de forma paramétrica respecto de una relación operacional, lo que permite reutilizar sus reglas con diferentes modelos de estado. La estructura se implementa mediante el sistema de *locales* de Isabelle/HOL.

Además, el framework incorpora mecanismos de automatización destinados a reducir el esfuerzo de prueba. Los autores aplican estas técnicas a distintos ejemplos y demuestran, con la IL, la presencia de vulnerabilidades de *double free*, incluida una vulnerabilidad de la biblioteca cURL.

Este es el trabajo más cercano a esta tesina en cuanto al sistema lógico estudiado. Los dos desarrollos mecanizan la IL, definen una semántica operacional y usan un asistente de pruebas para verificar que los razonamientos deductivos tienen respaldo semántico.

Su distinción está en el objetivo y nivel de abstracción. IsaBIL busca proporcionar un entorno semi-automatizado para verificar ejecuciones reales, incluso cuando no se tiene el código fuente. Por esa razón, tiene que representar instrucciones, registros, palabras de máquina, saltos y detalles de arquitecturas concretas. Esta tesina usa un lenguaje más chico para estudiar directamente la estructura metateórica de la IL y la ISL.

La diferencia más importante se encuentra en el razonamiento espacial. IsaBIL no incorpora actualmente la SL ni la ISL. Los autores explican que la memoria de BIL se representa con un historial de actualizaciones y que esa representación no es directamente compatible con la separación del *heap*, por lo que dejan la ISL como una extensión futura.

Automatización del razonamiento espacial con VeriFast

Jacobs et al. [12] presentan VeriFast, una herramienta de verificación para programas C y Java basada en SL y ejecución simbólica. El usuario anota los programas con contratos, predicados espaciales y otros elementos auxiliares, mientras que la herramienta ejecuta simbólicamente los comandos y verifica las obligaciones de prueba. VeriFast admite programas secuenciales y concurrentes y puede utilizarse para demostrar la ausencia de diferentes errores de ejecución, como accesos inválidos a memoria, desreferencias de punteros nulos y accesos fuera de los límites de arreglos. Su sistema de especificación permite representar permisos y predicados definidos por el usuario para describir estructuras dinámicas.

Este trabajo se relaciona con esta tesina porque los dos usan aserciones espaciales y conjunción separadora para razonar sobre fragmentos disjuntos de memoria. En ambos casos, los predicados inductivos permiten representar estructuras enlazadas sin enumerar individualmente todas sus celdas.

La diferencia principal reside nuevamente en la interpretación lógica. VeriFast usa SL para verificar que los programas respeten sus contratos y no produzcan determinados errores. La ISL formalizada en esta tesina usa sub-aproximaciones para demostrar que existen ejecuciones que alcanzan estados determinados, incluidos estados que representan accesos inválidos.

También difieren en el grado de automatización. VeriFast selecciona automáticamente los pasos de ejecución simbólica y aplica las reglas necesarias para procesar la estructura del programa. Aunque el usuario debe dar contratos, invariantes y predicados auxiliares, no construye manualmente un árbol de prueba para cada instrucción. En esta tesina, las derivaciones de `isl_triple` se construyen de forma explícita con los constructores del tipo inductivo. F^* y $Z3$ verifican los refinamientos y condiciones laterales, pero no eligen de manera autónoma los caminos de ejecución, las reglas ni las aserciones intermedias.

Por otra parte, VeriFast es una herramienta destinada a la verificación de programas concretos, mientras que el objetivo de la tesina es mecanizar el núcleo de una lógica y demostrar sus propiedades metateóricas. La comparación permite mostrar una posible línea de investigación futura, usar las definiciones y los teoremas mecanizados como fundamento de un procedimiento que automatice la construcción de derivaciones de ISL.

Resumen

La formalización propuesta en este trabajo se encuentra de alguna forma entre todos los desarrollos antes mencionados. Al igual que los tres primeros, se concentra en dar una representación mecanizada de la semántica, las reglas lógicas y sus propiedades. Al igual que VeriFast, incorpora aserciones espaciales y predicados sobre memoria mutable. Lo que lo diferencia del resto es la mecanización conjunta de la interpretación sub-aproximada de la IL y del razonamiento local de la SL.

El resultado no alcanza todavía el nivel de IsaBIL o VeriFast ni pretende analizar un lenguaje industrial. Su aporte consiste en dar un núcleo formal en el que las derivaciones de la IL e ISL son objetos explícitos y sus propiedades de *soundness*, localidad y descomposición por huellas son verificadas dentro de F^* .

6.3. Trabajo futuro

Detallamos a continuación algunas líneas de investigación que permitirían extender el desarrollo realizado.

Automatización de las derivaciones

Actualmente, los caminos de ejecución, las aserciones intermedias y los constructores utilizados en los árboles de prueba son seleccionados manualmente. Una extensión consiste en desarrollar un procedimiento que recorra la sintaxis del programa y construya automáticamente derivaciones de `isl_triple`. Este procedimiento podría calcular postcondiciones, seleccionar ramas de elección no determinista, descubrir caminos que llevan a errores y generar las obligaciones lógicas.

En el caso espacial, técnicas de bi-abducción [4] podrían usarse para inferir los recursos requeridos por una operación y los marcos que deben conservarse. Esto permitiría aproximar la formalización a una herramienta de análisis estático sin abandonar las garantías mecanizadas del núcleo lógico.

Extensión del lenguaje y del modelo de memoria

El lenguaje formalizado tiene las construcciones necesarias para estudiar las reglas centrales de la lógica, pero no incorpora llamadas recursivas, estructuras, arreglos ni aritmética general de punteros. Agregar estas características permitiría representar programas más cercanos a lenguajes de uso real. También podría desarrollarse un modelo de memoria con bloques de distintos tamaños, direcciones interiores, alineamiento y una representación más detallada de los valores almacenados.

Cada extensión requeriría agregar las reglas operacionales correspondientes y demostrar que estas nuevas reglas deductivas preservan el Teorema de Soundness.

Generación de trazas de error

El Teorema de Soundness retorna evidencia de que un estado final derivado en error tiene un estado inicial y una ejecución que lo alcanza. Una herramienta práctica podría transformar esa evidencia en una traza de error legible.

La traza podría indicar las instrucciones ejecutadas, las ramas no deterministas seleccionadas, los valores relevantes del *store* y las modificaciones realizadas sobre el *heap*. De esta manera, la demostración formal podría usarse también para producir una explicación concreta del error.

Evaluación experimental

La evaluación realizada en esta tesina es cualitativa y se concentra en la expresividad de la formalización. Una implementación con más automatización permitiría estudiar experimentalmente su comportamiento.

Podrían medirse los tiempos de verificación, el tamaño de las condiciones generadas, el consumo de memoria, la participación del solucionador SMT y la variación de los valores frente a programas de distintas dimensiones. También sería posible comparar diferentes estrategias de exploración y estudiar hasta qué punto la construcción de derivaciones sub-aproximadas permite controlar el crecimiento producido por las elecciones no deterministas y los bucles.

Concurrencia

Otra posible extensión es incorporar programas concurrentes. Esto requeriría incorporar construcciones de composición paralela y una semántica operacional que represente las posibles intercalaciones entre los pasos de los distintos hilos.

Raad et al. [18] presentan la Lógica de Incorrectitud de Separación Concurrente (CISL), que extiende la ISL al razonamiento sub-aproximado sobre programas concurrentes. Mecanizar las propiedades de concurrencia en F^* permitiría estudiar errores que no aparecen en programas secuenciales, como *race conditions*, *deadlocks* y fallas dependientes del orden de ejecución.

Bibliografía

- [1] Andrew W. Appel and Sandrine Blazy. Separation logic for small-step cminor. In Klaus Schneider and Jens Brandt, editors, *Theorem Proving in Higher Order Logics*, pages 5–21, Berlin, Heidelberg, 2007. Springer Berlin Heidelberg. ISBN 978-3-540-74591-4.
- [2] Yves Bertot and Pierre Castéran. *Interactive theorem proving and program development. Coq’Art: The Calculus of inductive constructions*. Springer Berlin Heidelberg, 01 2004. ISBN 3540208542. doi: 10.1007/978-3-662-07964-5.
- [3] David Brumley, Ivan Jager, Thanassis Avgerinos, and Edward J. Schwartz. Bap: a binary analysis platform. In *Proceedings of the 23rd International Conference on Computer Aided Verification, CAV’11*, page 463–469, Berlin, Heidelberg, 2011. Springer-Verlag. ISBN 9783642221095.
- [4] Cristiano Calcagno, Dino Distefano, Peter W. O’Hearn, and Hongseok Yang. Compositional shape analysis by means of bi-abduction. *J. ACM*, 58(6), December 2011. ISSN 0004-5411. doi: 10.1145/2049697.2049700. URL <https://doi.org/10.1145/2049697.2049700>.
- [5] Cristiano Calcagno, Dino Distefano, Jeremy Dubreil, Dominik Gabi, Pieter Hooimeijer, Martino Luca, Peter O’Hearn, Irene Papakonstantinou, Jim Purbrick, and Dulma Rodriguez. Moving fast with software verification. In Klaus Havelund, Gerard Holzmann, and Rajeev Joshi, editors, *NASA Formal Methods*, pages 3–11, Cham, 2015. Springer International Publishing. ISBN 978-3-319-17524-9.
- [6] Dino Distefano, Manuel Fähndrich, Francesco Logozzo, and Peter W. O’Hearn. Scaling static analyses at facebook. *Commun. ACM*, 62(8):62–70, July 2019. ISSN 0001-0782. doi: 10.1145/3338112. URL <https://doi.org/10.1145/3338112>.
- [7] Robert W. Floyd. Assigning meanings to programs. *Proceedings of Symposium on Applied Mathematics*, 19:19–32, 1967. URL <http://laser.cs.umass.edu/courses/cs521-621.Spr06/papers/Floyd.pdf>.
- [8] Mike Gordon. *From LCF to HOL: a short history*, page 169–185. MIT Press, Cambridge, MA, USA, 2000. ISBN 0262161885.
- [9] Matt Griffin, Brijesh Dongol, and Azalea Raad. IsaBIL: A Framework for Verifying (In)correctness of Binaries in Isabelle/HOL. In Jonathan Aldrich and Alexandra Silva, editors, *39th European Conference on Object-Oriented Programming (ECOOP 2025)*, volume 333 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 14:1–14:30, Dagstuhl, Germany, 2025. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. ISBN 978-3-95977-373-7. doi: 10.4230/LIPIcs.ECOOP.2025.14. URL <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ECOOP.2025.14>.
- [10] C. A. R. Hoare. An axiomatic basis for computer programming. *Commun. ACM*, 12(10):576–580, October 1969. ISSN 0001-0782. doi: 10.1145/363235.363259. URL <https://doi.org/10.1145/363235.363259>.

- [11] William Alvin Howard. The formulae-as-types notion of construction. In Haskell Curry, Hindley B., Seldin J. Roger, and P. Jonathan, editors, *To H. B. Curry: Essays on Combinatory Logic, Lambda Calculus, and Formalism*. Academic Press, 1980.
- [12] Bart Jacobs, Jan Smans, Pieter Philippaerts, Frédéric Vogels, Willem Penninckx, and Frank Piessens. Verifast: a powerful, sound, predictable, fast verifier for c and java. In *Proceedings of the Third International Conference on NASA Formal Methods*, NFM’11, page 41–55, Berlin, Heidelberg, 2011. Springer-Verlag. ISBN 9783642203978.
- [13] Quang Loc Le, Azalea Raad, Jules Villard, Josh Berdine, Derek Dreyer, and Peter W. O’Hearn. Finding real bugs in big programs with incorrectness logic. *Proc. ACM Program. Lang.*, 6(OOPSLA1), April 2022. doi: 10.1145/3527325. URL <https://doi.org/10.1145/3527325>.
- [14] Magnus O. Myreen and Michael J. C. Gordon. Hoare logic for realistically modelled machine code. In *Proceedings of the 13th International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, TACAS’07, page 568–582, Berlin, Heidelberg, 2007. Springer-Verlag. ISBN 9783540712084.
- [15] Peter O’Hearn. Separation logic. *Commun. ACM*, 62(2):86–95, January 2019. ISSN 0001-0782. doi: 10.1145/3211968. URL <https://doi.org/10.1145/3211968>.
- [16] Peter W. O’Hearn. Incorrectness logic. *Proc. ACM Program. Lang.*, 4(POPL), December 2019. doi: 10.1145/3371078. URL <https://doi.org/10.1145/3371078>.
- [17] Azalea Raad, Josh Berdine, Hoang-Hai Dang, Derek Dreyer, Peter O’Hearn, and Jules Villard. Local reasoning about the presence of bugs: Incorrectness separation logic. In Shuvendu K. Lahiri and Chao Wang, editors, *Computer Aided Verification*, pages 225–252, Cham, 2020. Springer International Publishing. ISBN 978-3-030-53291-8.
- [18] Azalea Raad, Josh Berdine, Derek Dreyer, and Peter W. O’Hearn. Concurrent incorrectness separation logic. *Proc. ACM Program. Lang.*, 6(POPL), January 2022. doi: 10.1145/3498695. URL <https://doi.org/10.1145/3498695>.
- [19] J.C. Reynolds. Separation logic: a logic for shared mutable data structures. In *Proceedings 17th Annual IEEE Symposium on Logic in Computer Science*, pages 55–74, 2002. doi: 10.1109/LICS.2002.1029817.
- [20] Nikhil Swamy, Guido Martínez, and Aseem Rastogi. *Proof-Oriented Programming in F**. F* Project, 2025. URL <https://fstar-lang.org/tutorial/proof-oriented-programming-in-fstar.pdf>.